IndiCare

Mobile IoT monitoring

Technical report and evaluation

Wearable sensing, native Android care and
secure self-hosted analytics

Team members	Nathan Ratichon	2362789
	Guannan Ge	2364229
	Jose Manuel Cortes Ceron	2686162
	Cheng Zhang	2361292
Tutor	Matilda Isaac	

Report date	30 October 2026
Repository	`https://github.com/deepdevjose/Older-Monitoring-Wearable`

Educational use only
This prototype must not be used for diagnosis, treatment or clinical decision making. Evaluation separates synthetic model exercises, Android checks and controlled physical bench observations.

Contents

1 Purpose and assessment scope **1**

2 Requirements and system architecture **1**

3 Hardware and data boundaries **3**

4 Mobile behaviour and programming structure **4**

5 REST integration storage and alert lifecycle **8**

6 Synthetic classification and model evaluation **10**

7 Security configuration and development evidence **12**

8 Testing measured outcomes and improvement **13**

9 Conclusion **14**

References **15**

A API contract and synthetic example **16**

B Reproduction installation and scenario evidence **17**

C Historical development images **21**

D Project team and development history **22**

E Demonstration evidence and evaluation boundaries **23**

F Current system architecture atlas **30**

1 Purpose and assessment scope

IndiCare combines an ESP32 wearable, the CareMonitor 1.0.0 Android gateway and a local REST service. Native adult and caregiver screens present readings, history, persistent alerts and saved synthetic classifications with confidence. Evidence separates current software checks, dated production-app campaigns and controlled physical-system execution; each result identifies the layer exercised.

The assessment covers patient and caregiver views, three class predictions, confidence, storage, alerts and measurable improvement. Mobile functionality and programming architecture carry 45% of the rubric. Requirements are traced to code, quantitative results and visible application behaviour using the versioned API and release records. Historical captures retain their original scope.[1, 2]

Classification uses synthetic educational labels from the repository CSV, without implying institutional dataset approval or clinical ground truth. A consented optical bench check is documented separately. Clinical records, induced falls and the separate SisFall experiment are excluded from the reported model results. Historical screenshots document development without publishing personal physiological values.

2 Requirements and system architecture

The primary actors are an adult account, corresponding to the rubric's patient role, and an explicitly authorised caregiver. An adult can approve several caregivers and revoke their access independently. Requirements follow a reading through provenance, validation, transfer, persistence, presentation and alert review. Table 1 associates these behaviours with implementation locations and the evidence available for each one.

Figure 1 distinguishes two implemented mobile and server paths. The care interface retrieves readings, historical metric aggregates and rule alerts. Learning insights, reached through authenticated Settings, submits synthetic feature vectors and displays saved classifications with confidence. Its encrypted records occupy a separate table; neither synthetic model exercises nor confidence scores become device readings or care notifications. This separation supports assessment of the model while preserving the diagnostic source of wearable observations.

A local FastAPI service keeps Python validation, models and account authorisation together. Central inference avoids distributing model updates to every phone but depends on server availability. SQLite offers transactional episode creation with modest deployment complexity; its single writer architecture suits the evaluated scale. Native Android Java provides Bluetooth and foreground service access without a cloud service.

The wearable ESP32 path supplies device diagnostics over Bluetooth Serial Port Profile to the Android gateway. Source labels distinguish diagnostics from synthetic assessment data. Server inference rejects device sources because synthetic body temperature is not interchangeable with the available surface temperature measurement. HTTPS protects phone to server communication. Before sensor transmission, the app and wearable prove knowledge of a private per-board key through fresh, role-separated HMAC challenges. The board admits one current phone connection, filters events by connection identity and releases ownership on disconnection. Encrypted SSP without device input removes USB confirmation; resistance to an active pairing relay is not established. Account and resource authorisation apply independently of either transport.

Evaluation checks duplicate suppression, authorised access, persistent episodes and explicit unknown states. Timing measurements identify their transport and observation boundaries; no numeric delivery guarantee is inferred from an average.

Table 1. Requirements traced to implementation and distinct executed evidence.

ID	Requirement and component	Verification and scope	State
R1	Adult and caregiver views; MainActivity	Role navigation, wearable guard and real approved-caregiver views	E/D/F/Q
R2	Reading storage and retrieval; readings API, SQLite	Idempotency/order tests; physical readings retained and fetched through native views	E/Q
R3	Tappable history; history API, MetricHistoryChartView	Day/Week/Month fixtures; actual surface-history views under both roles; aggregation/access tests	E/F/Q
R4	Three classes and confidence; ModelService, LearningActivity	Synthetic exercises saved and read via real API under both roles; model metrics tested separately	E/F/Q
R5	Stored alerts and private inbox	Rule/recovery and independent-review tests; native caregiver help notice and posting trials	E/Q
R6	Missing, invalid and disconnected states	Validation/relay tests and native unavailable states; physical outage recovery not measured	S/E/F/Q
R7	Roles, consent and credentials	TLS, authorisation and revocation tests; real request/approval before caregiver access	E/Q
R8	Modular app and accessible navigation	JVM logic, labelled tabs and enlarged-text fixture; Activities retain view/controller coupling	E/D/F
R9	Version control and attribution	Supplied contribution matrix and artefact links identify tasks; dated commits and build hashes record integration	S
R10	Demonstration and improvement	Learning exercises, matched retry comparison and measured native-posting improvement	E/F/Q

S: source inspected; E: backend/JVM software executed; D: dated production-package phone checks; F: current isolated preview fixture. Q: 30 October system QA on a physical S22 with real accounts and HTTPS, using production Java/resources in a separate application ID and database. Q is not acceptance of the installed production APK or a two-handset deployment.

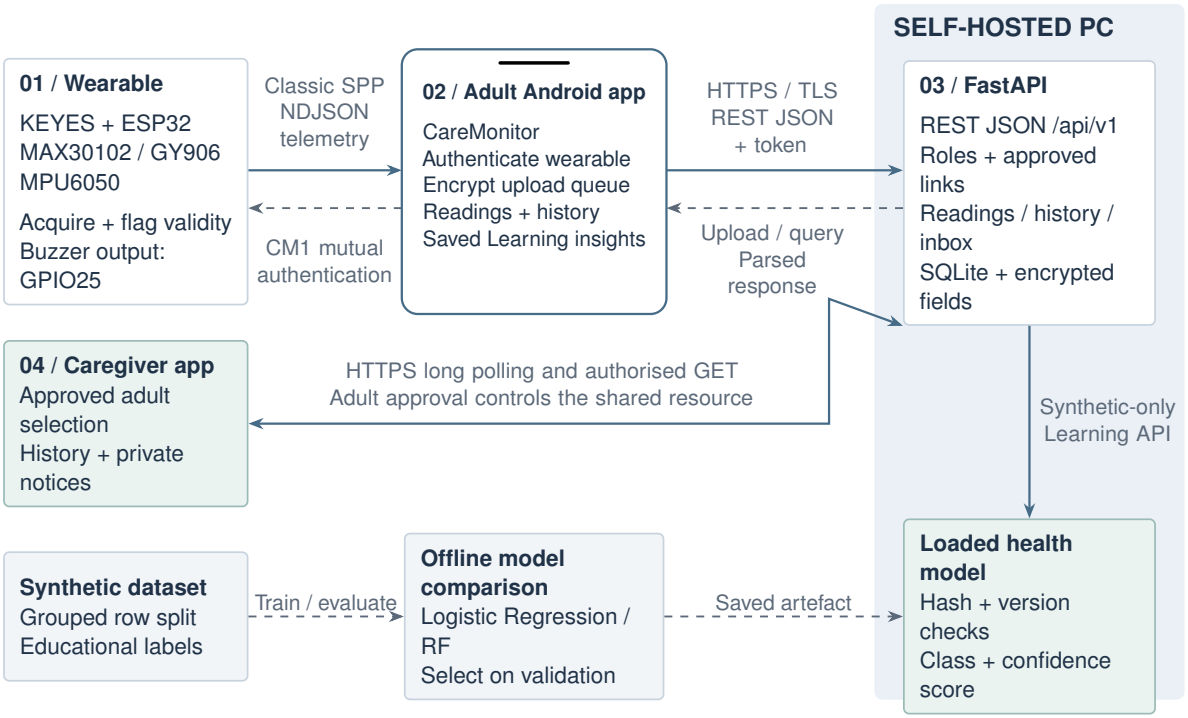


Figure 1. Complete system and its separate care and synthetic-learning paths. Physical SPP, API-backed native views and saved synthetic confidence were exercised in the controlled code-20 campaign; model training was evaluated independently. USB forwarding carried HTTPS in that campaign; reachable LAN deployment uses the same API. The two roles were exercised successively on one S22.

3 Hardware and data boundaries

The current wearable comprises a KEYES ESP32 carrier, MAX30102, MPU6050, GY906 and buzzer on GPIO25. Microphone and light modules have been removed. Figure 19 maps the shared bus and GPIOs; Figure 18 explains each component. Optical contact and validity flags distinguish usable estimates from unavailable readings; missing data cannot prove that the wearable is off the body. Surface temperature cannot establish fever, and these modules do not measure hydration. The buzzer output is present without autonomous fall-trigger logic.

The optical pipeline uses a rolling sample window and validity flags; these do not establish physiological accuracy. During the earlier 27 October network commissioning, an application-only upload was hash verified after a private flash backup, retaining the bootloader and partition table. The subsequent boot associated with the modem and synchronised NTP. A 65 second observation received 63 valid JSON records at 0.999 Hz, with motion and temperature modules online throughout and no resets. Fifty nine records contained clock timestamps; the largest observed clock error was 0.55 seconds. Optical readings remained unavailable without contact. Only aggregate status metadata was retained. Verified software identity and communication do not establish calibration or clinical validity.[19]

Figure 2 preserves the supplied assembly photograph as a historical record; the updated wiring diagram identifies the current modules. Calibration and runtime identity require separate evidence. Appendix C retains dated serial/HTTP captures, while Appendix F details current protocols, permissions, actors, classes, storage and security.

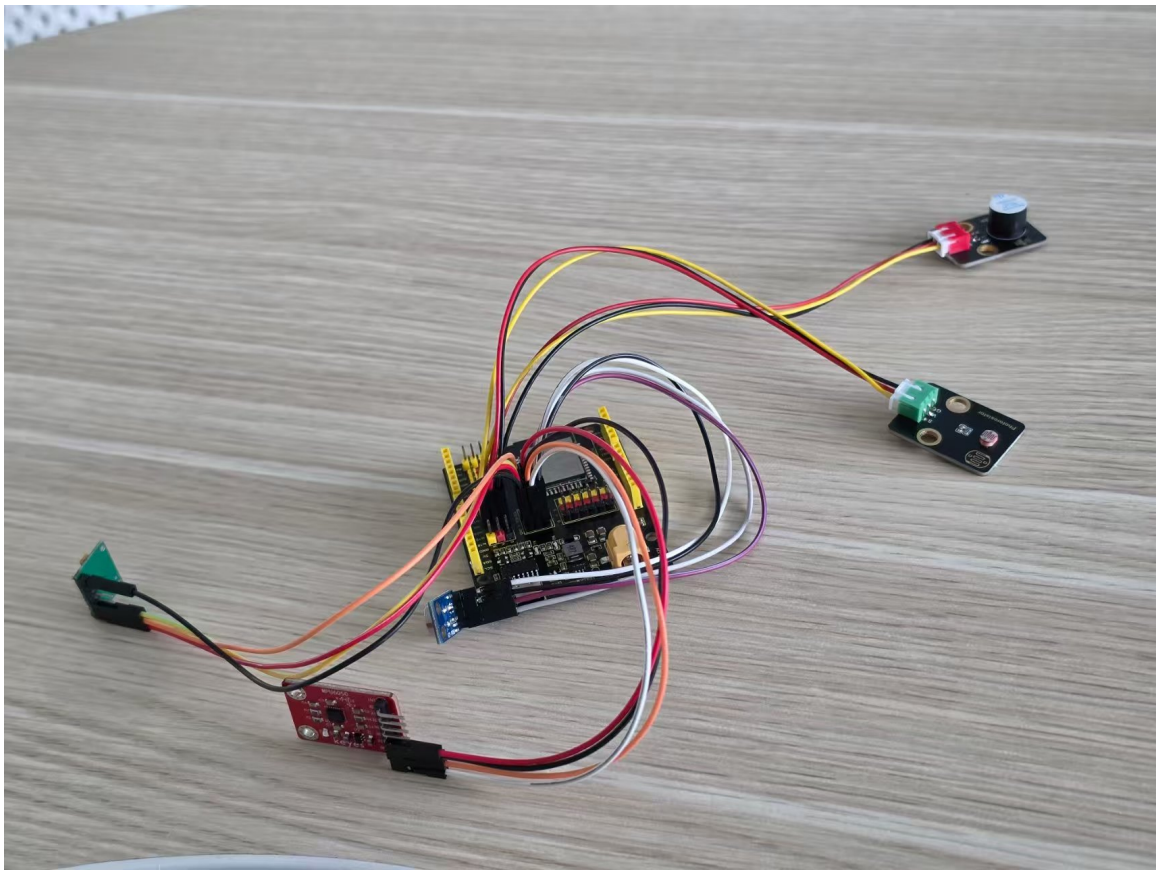


Figure 2. Earlier physical prototype photograph supplied by the team. The KEYES carrier and sensor modules document the bench assembly before the microphone and light sensor were removed. The current four-module wiring map is Figure 19; this older photograph is not presented as a photograph of the revised arrangement.[7]

4 Mobile behaviour and programming structure

The Android application implements registration, login, adult and caregiver navigation, reading summaries, alert history and settings. Registration introduces the role before collecting name, email and password in four progressive steps. Server configuration belongs to authenticated Settings, keeping technical connection details outside the initial account form. Wearable setup is restricted to the adult role; caregivers use the relationship request and approval workflow. Separate accounts and server enforced revocation prevent shared credentials or locally hidden controls from becoming the access model.

Progressive registration presents one category of information at a time and preserves the draft when moving backwards. The primary action remains available while inputs scroll, including at enlarged text. Passwords are initially masked with an explicit visibility control. Role selection determines the next task: adults configure a wearable; caregivers request authorised access. Instrumentation checks these behaviours without establishing user preference or reduced cognitive burden.

The English interface organises Today around each role. Adults reach help, their next reminder and readings; caregivers select an approved person and review pending notices, readings and shared location. Updates and Care retain stable navigation. Settings contains Reading details, replacing the technical Recent readings card on the home screen. Large controls and metric histories support progressive disclosure. Live text and freshness update in place to preserve focus. Brief connection motion respects Reduce motion, system animations and battery saving without interpolating sensor values. Figure 3 illustrates both roles with fictional profiles. These decisions are design hypotheses rather than findings from an older adult usability study. The dated code 13 campaign passed 709 onboarding checks, including enlarged registration text; current device scope is recorded separately.[16, 5, 6]

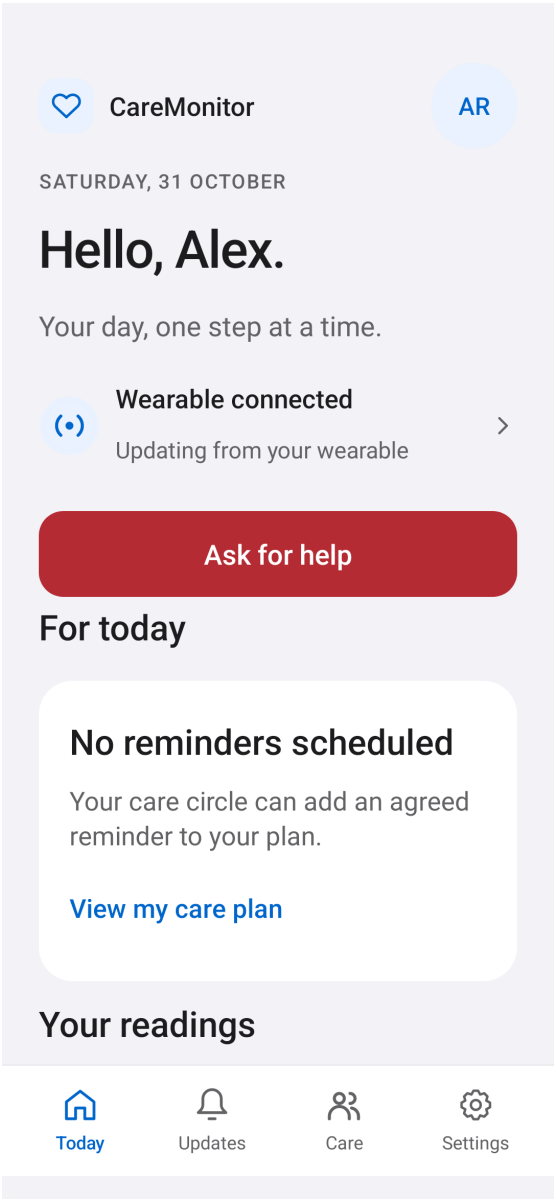
W3C and Apple accessibility guidance motivate readable text, predictable navigation and controllable motion. They provide a design rationale; no standards conformance or older-adult usability study is claimed.[21, 22]

Figure 4 shows a layered, MVC inspired arrangement. MainActivity coordinates care screens and metric navigation; LearningActivity handles synthetic exercises. Both combine view composition with presentation control. MetricHistoryChartView draws timestamped aggregates, while ReadingSummary, LiveSampleFreshness and AlertInbox encapsulate reusable behaviour. ApiClient and ApiSession manage requests and account state, SecureStore protects persistence, and SensorTransport separates Bluetooth implementations. This is partial separation of concerns rather than complete MVVM. Android's guidance informs the boundary between orchestration and reusable data responsibilities.[15]

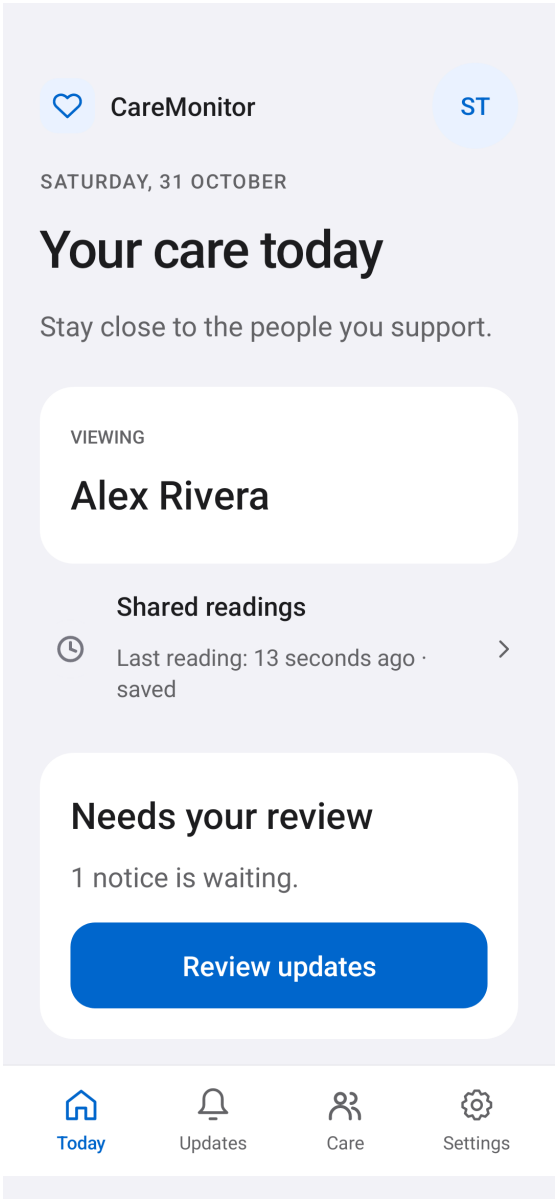
Bluetooth acquisition and queued relay use dedicated services and workers. Network tasks run through an executor; interface changes return to the main thread. Request, session and lifecycle guards reject stale callbacks. Table 2 identifies actual methods. Notification monitoring requires explicit activation and a visible foreground notification. Preview instrumentation preserves production account state; the separate system campaign uses real controlled accounts.

Tapping any of five current metric cards opens its mounted history chart, including when the current value is unavailable. Adult and authorised caregiver views use the selected person's resource. Day, Week and Month represent rolling 24 hour, seven day and thirty day windows; earlier and later navigation changes the interval. Encrypted hourly rollups retain valid counts, sums and extrema. Weighted means preserve unequal sampling density; invalid observations remain gaps. Boundary hours are read exactly, and incomplete legacy indexing is disclosed. These properties are covered by backend tests, separately from physical sensor accuracy.

Learning insights presents editable synthetic features and Normal, Warning and Critical CSV examples. An adult can classify and save; an approved caregiver can review that person's retained exercises. Each result shows its class, model score, timestamp and source. This implements mobile presentation of confidence without silently substituting GY906 surface temperature for the model's body temperature.



(a) Older adult during authenticated physical reception.



(b) Caregiver: selected person and notices to review.

Figure 3. Code 20 production-view captures on S22 / Android 13, using real HTTPS test accounts in an isolated package/database and 100% text. The adult is captured during physical SPP reception; the caregiver is captured later after an approved link, reminder and controlled help event. These are successive observations, not simultaneous phones or in-memory fixtures. Full provenance and additional views appear in Appendix E.1.

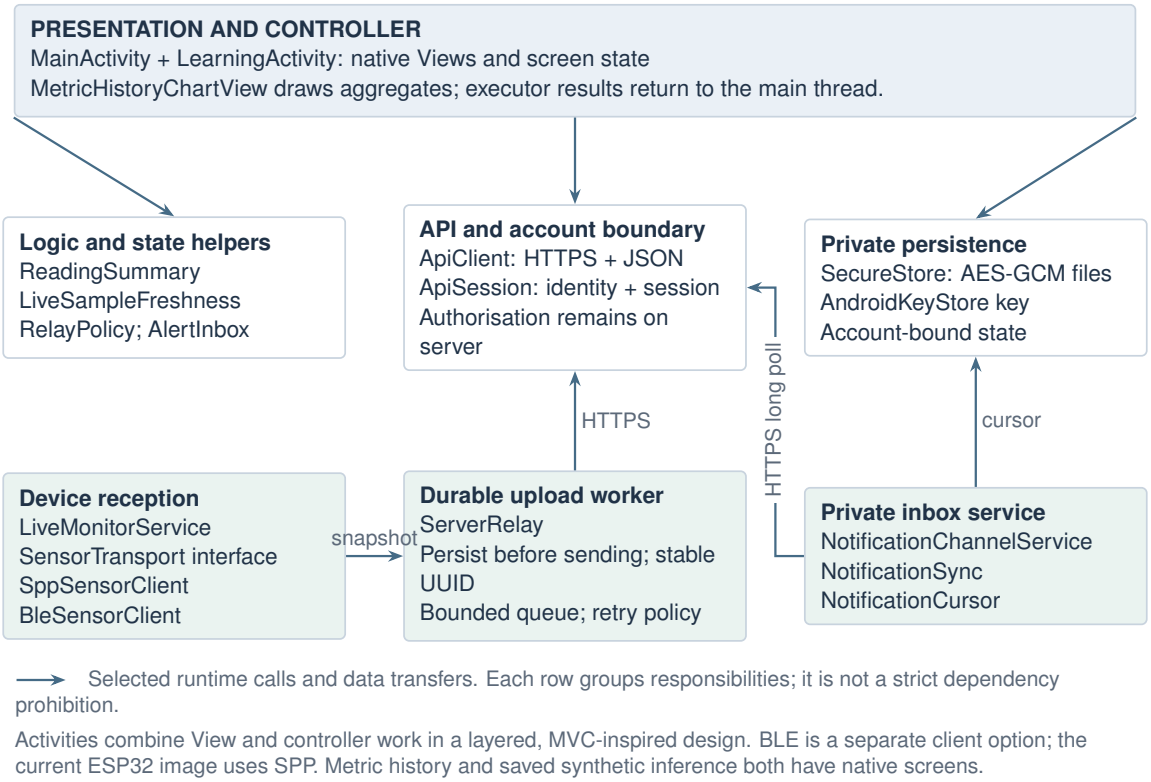


Figure 4. Actual Android organisation. Reusable transport, upload, notification, HTTPS and state helpers support the activities. Source separation and JVM tests do not alone establish Android lifecycle, accessibility, Bluetooth or background-delivery acceptance.

Table 2. Concrete responsibility boundaries in the Android source. Methods are traceable in the named canonical Java source files.

Responsibility	Class and representative methods	Ownership and limit
Care presentation	MainActivity: `render()`, `history()`, `openHistory()`	Builds native views and handles navigation; view and controller duties remain combined
Asynchronous state	MainActivity: `loadHistory()`, `historyCurrent()`	Executor performs requests; main-thread callback verifies account, person, period and request identity
Synthetic exercises	LearningActivity: `load()`, `classify()`, `render()`, `current()`	Separate screen, role guard and lifecycle checks; retains its own view/controller composition
Chart rendering	MetricHistoryChartView: `setHistory()`, `onDraw()`	Draws supplied aggregates and supports selection; does not fetch readings or run inference
Account and storage	ApiClient; ApiSession: `load()`, `save()`, `clear()`; SecureStore	HTTPS/JSON boundary, expiring account state and encrypted persistence
Acquisition and policy	SensorTransport, SppSensorClient; ServerRelay; ReadingSummary, LiveSampleFreshness, AlertInbox	Radio access, queued upload and reusable logic remain outside screen composition

Table 3. Role behaviour in the current release. Server checks enforce access; the Android UI also restricts wearable controls to the older adult.

Action	Older adult	Approved caregiver
Read current overview and alert history	Own account	Linked adults only
Submit own sensor snapshots	Own account	Cannot impersonate adult
Approve or revoke caregiver	Own circle	Cannot self approve
Review alert	View own history	Own acknowledgement only
Account and connection settings	After sign in	After sign in
Guided wearable connection	Own wearable only	Controls unavailable
Metric chart and period navigation	Own retained readings	Linked adult's readings
Synthetic AI class and confidence	Add and review own exercises	Review linked adult's exercises
Registration accessibility	200% layout checked	200% layout checked

5 REST integration storage and alert lifecycle

The API uses the prefix `/api/v1`. Users authenticate through account endpoints and send a Bearer session token. Readings carry a stable sample identifier, capture timestamp, source and structured payload. Pydantic contracts reject malformed or unsupported inputs, while account and resource checks enforce access. Appendix A records the contract. Figure 13 traces a retained physical sample through the relay, server commit and native views: the controlled campaign stored 76 device readings and drained its outbox. Both adult history and approved-caregiver views used real HTTPS responses.[3, 25]

The phone queue retains snapshots during temporary upload failure. Reusing a sample identifier makes a retry idempotent: a lost response creates neither another sample nor another alert. The server writes reading state, alert state and private notification entries transactionally. SQLite isolation lets readers observe committed changes; it does not guarantee delivery to a disconnected handset.[18] Figure 5 traces a qualifying upload through storage and fanout to two approved caregivers.

The outbox stores each item before transmission and removes it after acknowledgement. Its capacity is two hundred items, so the design does not imply lossless indefinite disconnection. Items retain account and endpoint ownership to prevent relay across sessions. Authentication, permission and validation errors pause sending; transient failures use retry and backoff. Policy tests exercise these decisions. Actual process termination, AndroidKeyStore persistence and subsequent physical delivery form a different device execution boundary from those tests.

Synthetic rule episodes require three recent valid samples with distinct timestamps separated by one to thirty seconds. Sustained abnormal input creates one unresolved episode per signal. Further abnormal samples update its evidence without repeatedly creating the same episode. Three eligible observations no longer triggering the rule recover it. Missing contact, disconnected sensors and invalid values cannot close an episode because unusable evidence does not establish recovery. Figure 6 makes these transitions explicit.

Acknowledgement records caregiver specific review rather than recovery. One caregiver's action does not suppress another caregiver's history. Delivery acknowledgement is also distinct from review. A persistent private inbox and authenticated long polling support recovery after interruption. The 26 October benchmark measures server processing; Appendix E.1 separately measures native notification posting.

The synchronous classification routes run through FastAPI's thread pool.[13] Bounded inference slots and per account limits constrain work. A loaded model is cached; an unavailable or busy model returns an explicit error rather than a fabricated normal result. The original classification route remains stateless. The learning route additionally stores an encrypted synthetic result with an idempotent exercise identifier and

thirty day retention. Resource authorisation is checked again before committing. Model confidence does not drive the separate rule episodes.

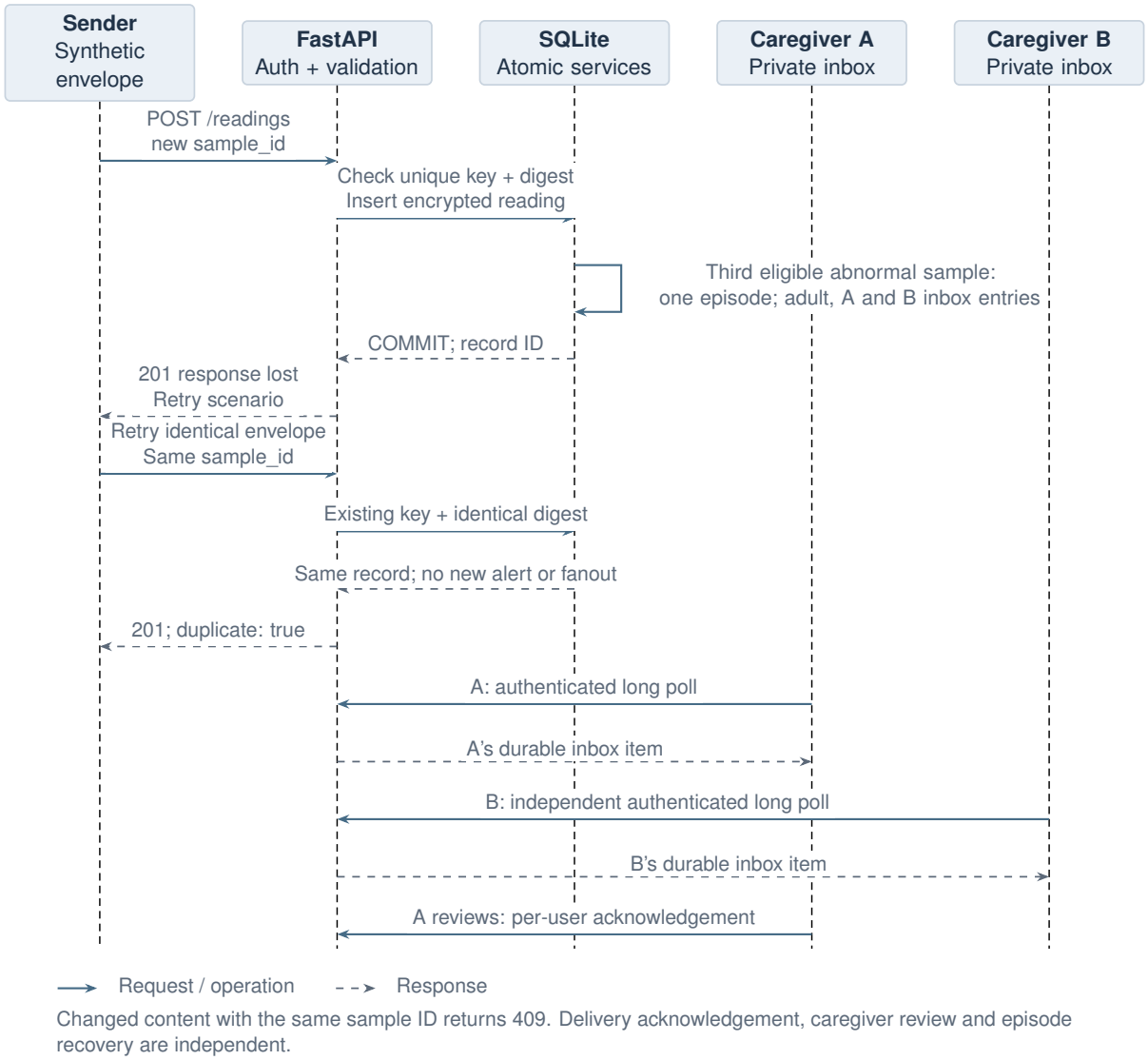


Figure 5. Atomic reception, safe retry and independent recipient fanout. The reading, any eligible rule episode and recipient inbox entries commit together; an identical retry returns the existing record. This contract was verified with loopback clients; simultaneous delivery to two physical phones is not claimed.

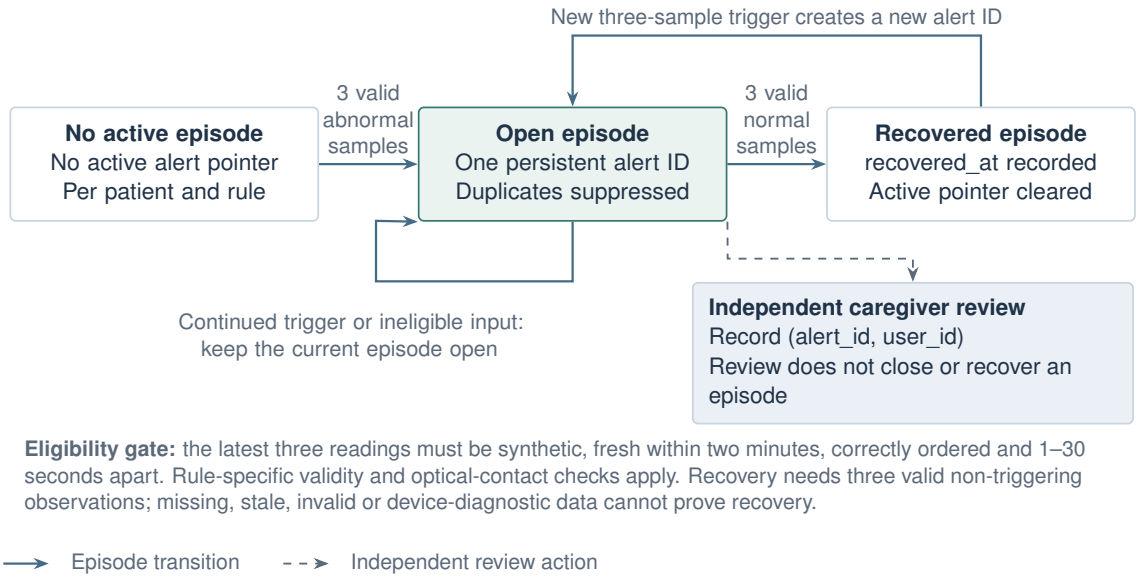


Figure 6. Implemented educational episode lifecycle in alerts.py. A recovered alert remains in history; only its active pointer is cleared. "Recovered" refers to a synthetic rule episode, not a measured health outcome.

6 Synthetic classification and model evaluation

The experiment reads the 60,000 row repository CSV using pandas. It retains three numeric features: synthetic heart rate, oxygen saturation and body temperature. Identifiers, other measurements and alert columns are excluded from the feature matrix. Missing, nonnumeric or nonfinite values fail preparation instead of being interpreted as healthy. Targets count abnormal flags across the three alert columns: zero gives Normal, one Warning, and two or three Critical. These constructed educational categories have no independent clinical ground truth.[4]

Repeated feature tuples could place identical examples in training and evaluation. The training script groups the 18,939 unique feature tuples before a stratified split with seed 42. Every occurrence of one tuple stays in one partition. The resulting counts are 36,090 training, 12,143 validation and 11,767 test rows. This controls exact tuple leakage without creating external validation. The test partition is a local holdout, not an assessor controlled hidden test set.

Two introductory supervised classifiers are compared. Logistic regression uses StandardScaler fitted within its training pipeline, balanced class weights and at most 2,000 iterations. Random Forest uses 120 trees, maximum depth fourteen, minimum leaf size two and balanced class weights. Validation macro F1 selects the model, with Critical recall as tie breaker. Test results are reported after selection rather than used to select. Random Forest reaches validation macro F1 of 1.0000 against logistic regression's 0.7534 and is saved for endpoint inference.

Table 4 reports accuracy and macro precision, recall and F1. Macro averaging weights categories equally; Critical recall also exposes missed critical classifications.[14] Figure 8 presents both confusion matrices with true classes as rows. Logistic regression misclassifies 905 of 3,857 Critical examples and assigns Warning to 420 of 3,366 Normal examples. Random Forest makes no observed holdout errors. Table 5 defines the corresponding rates and denominators.

The perfect forest result reflects reconstruction of deterministic synthetic boundaries rather than medical reliability. Robustness to another generator, sensor noise or distribution shift is outside this experiment. Confidence is the largest predict_proba score, rounded to six decimals. It is not calibrated against real outcomes and cannot represent the probability of disease or emergency. The inference contract requires fresh synthetic features within recorded training ranges. Before deserialisation, the local model file is hash checked and its scikit learn version compared with metadata. This mechanism does not make untrusted model files safe to load.

Logistic regression offers an interpretable baseline, but linear decision surfaces cannot readily represent every disconnected region created by combinations of threshold flags. A tree ensemble represents those combinations more directly. This interpretation is consistent with the matrices without establishing a unique explanation for each error. Reporting all classes prevents a favourable average from concealing weak Critical recall. The result is a reproducible local model comparison whose synthetic targets and dataset provenance delimit its interpretation.

Evaluation keeps feature preparation, model selection and final reporting distinct. Scaling is learned from training data rather than the complete dataset, and validation guides selection before the holdout is inspected for the reported comparison. Grouping prevents duplicate feature tuples from inflating performance through exact repetition across partitions. It does not remove the shared rule generator behind all three partitions. The resulting high score is therefore informative about reconstruction of the supplied synthetic task, while generalisation to a new population is a separate scientific question.

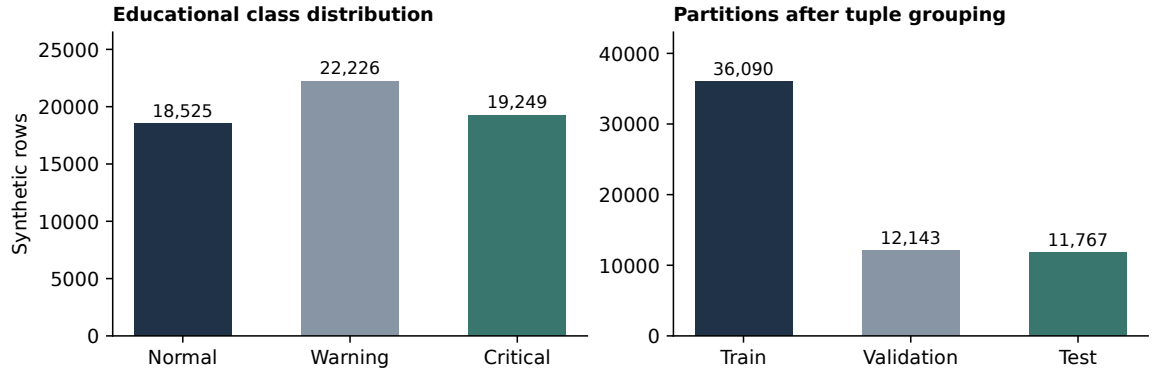


Figure 7. Class balance and row partitions from the saved training metadata. Splitting operates on unique feature tuples, so row proportions differ slightly from 60/20/20. These labels are educational categories.

Table 4. Local validation and test performance. P, R and F1 are macro averages; all values are percentages. Validation selects the model.

Model	Partition	Accuracy	P	R	F1	Critical R
Logistic regression	Validation	75.15	75.18	75.53	75.34	73.85
Random Forest	Validation	100.00	100.00	100.00	100.00	100.00
Logistic regression	Test	75.35	75.85	76.46	76.11	76.54
Random Forest	Test	100.00	100.00	100.00	100.00	100.00

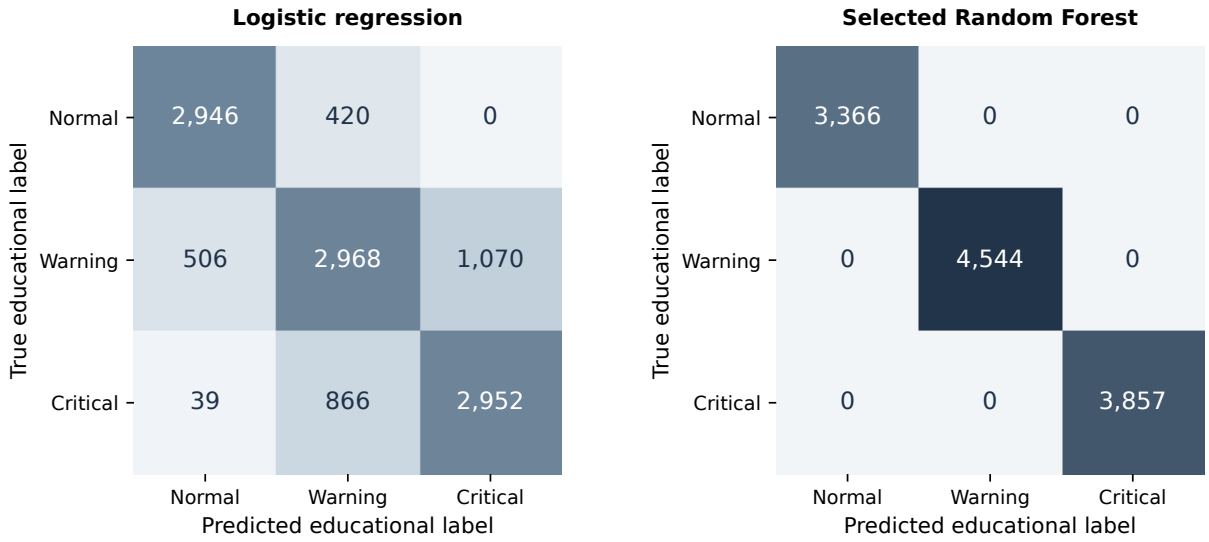


Figure 8. Confusion matrices for 11,767 locally held out synthetic rows. The selected forest reconstructs the rule derived labels perfectly on this split. This is neither clinical validation nor an assessor hidden test.

Table 5. Classification errors and alert observations with explicit denominators. Classifier rates do not measure phone delivery failures.

Measure and denominator	Logistic regression	Random Forest
Missed Critical classification: predicted other class / all true Critical	905/3,857 = 23.46%	0/3,857 = 0%
False Warning on Normal: predicted Warning / all true Normal	420/3,366 = 12.48%	0/3,366 = 0%
False Critical on noncritical: predicted Critical / true Normal or Warning	1,070/7,910 = 13.53%	0/7,910 = 0%
Server episode campaign	20/20 created; 0/20 duplicate episodes	
Missed Critical alert at caregiver phone	Outside server benchmark scope	
False Warning delivered to caregiver	Outside server benchmark scope	

7 Security configuration and development evidence

Accounts use Argon2id password hashes and random session tokens with hashed server representations. Resource access requires the correct role and an active approved relationship to the adult. Invitation codes request access without independently authenticating caregivers or disclosing readings. Expiry, limited attempts and adult approval constrain guessing. Argon2id follows established storage guidance; its inclusion is an implementation property rather than evidence of a comprehensive security audit.[17]

HTTPS verifies certificate names. Sensitive server fields use authenticated AES GCM encryption, while Android protects stored state through its KeyStore backed store. Identifiers, timestamps and event types are visible database metadata. Private configuration, database state and signing material are excluded from version control. The checked in server certificate is public trust material, and backups use a passphrase with restoration invalidating sessions. These distinctions define the protection actually provided.

The contribution matrix in Appendix D records the allocation supplied by José: Nathan supported exploratory fall trials, sensor checks and reporting; Guannan and Cheng developed the interface; José developed the backend and refined firmware and JSON. Guannan also developed the initial integrated firmware, while Cheng tested individual sensors with Nathan. All four shaped the concept, sensor rationale, architecture and protocols. Repository commits establish integration chronology; associated artefacts locate deliverables without inferring individual authorship from the shared Git identity.[12, 11]

8 Testing measured outcomes and improvement

Current validation passes 229/229 tests and 3,276 assertions, with 24/24 checks over loopback HTTPS. The isolated preview campaign exercises identical production Java and resources on Samsung S22 SM-S901E with normal and enlarged text; its fictional state does not exercise real accounts or a server. Targeted suites cover history aggregation and persisted learning, including authorisation, revocation, retention and idempotency. Table 6 distinguishes these results from the dated baseline: 88 Python tests, 2,830 JVM assertions, seventeen HTTPS checks and 709 onboarding checks. Software and fixture campaigns do not establish complete caregiver delivery.[5, 8, 6]

A separate 27 October physical campaign passed 207 protocol checks, including mutual authentication, rejection of stale proofs and an independent radio denied access while the owner remained connected. The normal code 17 application then received 138 samples, reached zero pending uploads and obtained server confirmation over Wi-Fi. Database counts increased from 156 to 295 readings without creating an account. These dated transport results are not reassigned to a later interface revision and do not establish caregiver notification arrival.[20]

A 30 October campaign executed 119 system checks using production Java and resources in an isolated Android package and HTTPS database. Physical SPP supplied 76 retained device readings with zero pending uploads. Thirty phone overview requests had p95 77.04 milliseconds. Native caregiver notification posting improved from 21/30 to 30/30 within ten seconds after limiting visible cards to twelve, preserving inbox history. Final posting p95 was 1,094.37 milliseconds. These screen-on, same-handset USB measurements do not establish two-phone, audible or locked-screen delivery. Appendix E.1 retains raw trials, source hashes and native captures.[25]

The 26 October baseline benchmark uses disposable databases and an in process ASGI TestClient. Twenty five warm classification requests produce mean response time 17.34 milliseconds and p95 19.87 milliseconds. The first request takes 1,059.29 milliseconds, including lazy model loading. Twenty synthetic pulse episodes produce p95 9.09 milliseconds for third qualifying ingestion and 15.02 milliseconds from before ingestion until an authenticated inbox read completes. All twenty retry and sustained sample checks retain one unresolved episode. These timings were not remeasured for the history revision.[9]

These baseline measurements exclude TLS, network transport, the preceding sampling interval, long polling and Android rendering. They establish local server processing, separately from the later handset notification campaign. Table 7 records boundaries and sample counts. Likewise, classifier false warnings and missed Critical labels describe holdout prediction errors, not complete application delivery failures. Keeping these denominators distinct makes both performance and reliability claims interpretable.

The forest improves test accuracy by 24.65 percentage points and macro F1 by 23.89 points over logistic regression. A separate chronological backend experiment compares the initial server at commit 6757c0a with the evaluated implementation. Both receive twenty distinct synthetic samples twice, representing a lost response and retry. The initial server stores forty rows; the evaluated server stores twenty, with every retry reusing its existing identifier. This removes twenty duplicate rows in the matched campaign. The adapter script documents the different contracts. The result demonstrates improved persistence semantics rather than faster operation or mobile inference integration.[10]

Scenario coverage distinguishes deterioration from malformed input. Valid synthetic sequences progress from normal values to sustained abnormal conditions and recovery. Missing fields, nonfinite values, stale timestamps and absent optical contact instead cause rejection or preserve unknown state. Retry policies and lost responses exercise network failure decisions; unlinked and revoked caregivers exercise authorisation. Appendix B specifies the measurement protocol and separates executable software scenarios from physical transport and operating system behaviour.

The campaigns support persistent episodes, reproducible models, native history/confidence views and physical acquisition within recorded boundaries. Screen reader accessibility, clinical sensor validity and unattended two-phone delivery need further evaluation.

Table 6. Current revision and dated baseline verification. Counts are not added because the units and campaigns differ.

Evidence	Result	Scope and exclusions
Current Python suite	229/229 tests	Current revision software record; units are tests
Current JVM suite	3,276 assertions	Pure Java checks; not Android runtime acceptance
Metric history suite	22 tests passed	Weighted means, validity, gaps, boundaries, retention and consent
Learning record suite	14 tests passed	Synthetic persistence, idempotency, authorisation and revocation
Current Android scope	Code 20	Installed code 20: identity and readonly HTTPS check; 7,073 isolated preview assertions; 119 actual-system assertions in an isolated package. Main-app onboarding skipped to preserve the existing session
Current HTTPS scope	24/24 checks	Loopback checks; distinct from physical transport
30 October system QA	119 checks	Real SPP, HTTPS, controlled accounts and native views; 76 retained readings, zero pending uploads. Final native posting 30/30 within ten seconds on one screen-on S22
27 October Bluetooth	207 protocol checks	Exclusive authenticated link; code 17 received 138 samples with zero pending. Dated proof; current APK and caregiver delivery excluded
Baseline Python/JVM	88 / 2,830	26 October records: Python tests / JVM assertions
Baseline HTTPS	17/17 checks	Loopback TLS, fictional accounts; no phone or Bluetooth
Baseline APK and UI	Code 13; 709 checks	Installed on Samsung SM-S948B, Android 17; registration at 200% font scale. APK SHA-256 `417c12ef...206378`
Baseline benchmark	20/20 episodes	Temporary databases; no duplicate retry or sustained episodes

Table 7. 26 October baseline server processing measurements in milliseconds. Percentiles use linear interpolation; no transport or Android latency is included.

Timed operation	N	Mean	p50	p95
Warm authenticated classification	25	17.34	16.85	19.87
Third qualifying reading and episode commit	20	7.08	6.95	9.09
Third reading through authenticated inbox GET	20	12.24	12.02	15.02
Cold classification including model load	1	1,059.29	—	—

The first two episode samples were already stored. Inbox timing uses the fictional adult's inbox; caregiver fanout is verified separately by the HTTPS integration fixture.

9 Conclusion

IndiCare integrates wearable acquisition, consent based Android care and self-hosted persistence. Random Forest improves synthetic holdout accuracy by 24.65 percentage points and macro F1 by 23.89 points over logistic regression. Idempotency reduces forty retry rows to twenty unique records. The final controlled system campaign retains 76 physical readings, drains the upload queue and posts 30/30 caregiver notifications within ten seconds. Both roles display retained history and saved synthetic confidence. These measured achievements establish the educational system within its documented execution boundaries, without implying clinical effectiveness or unattended emergency response.

References

[1] IOT308TC. *Assessment 2 AI Enabled Mobile IoT Healthcare Monitoring Application*. Supplied assessment brief and marking rubric, `rubrik.docx`. 3,000 words ±10%; eight weighted criteria.

[2] IndiCare project. *CareMonitor 1.0.0 implementation and acceptance status*. Canonical `README.md`, `docs/STATUS.md`, `docs/SECURITY_ETHICS.md` and `release/FIRMWARE_DIAGNOSTIC.json`. `https://github.com/deepdevjose/Older-Monitoring-Wearable`.

[3] IndiCare project. *Versioned API and implementation*. `docs/API.md`, `server/caremonitor/api.py`, `schemas.py`, `alerts.py`, `db.py`, `routes_ml.py` and Android source in `mobile/app/src/main/java/org/caremonitor/demo/`.

[4] IndiCare project. *Synthetic health model training and evaluation*. `tools/train_models.py`; `models/synthetic-health-v1/metadata.json`. Seed 42; scikit learn 1.8.0. Snapshot included as `evidence/model-metadata.json`.

[5] IndiCare project. *Release software validation, Android build and device interface checks*. `evidence/VALIDATION.json`, `ANDROID_BUILD.json`, `ANDROID_UI_EVIDENCE.json`, `pytest-current.xml` and `jvm-current.txt`, 26 October 2026. APK code 13; 709 checks on Samsung SM-S948B with Android 17.

[6] IndiCare project. *Role based interface, metric history and synthetic learning revision*. `evidence/CURRENT_REVISION.tex`; implementation in `history.py`, `learning.py`, `MainActivity.java` and `LearningActivity.java`, 30 October 2026. Test and device scope is distinguished from baseline snapshots.

[7] IndiCare team. *Physical prototype assembly photograph*. Supplied 26 October 2026; reproduced as `figures/hardware_prototype.jpg`. Original source path and SHA-256 recorded in `evidence/hardware_photo.json`.

[8] IndiCare project. *Isolated HTTPS integration evidence*. `evidence/current/INTEGRATION_EVIDENCE.json`. Current record: 24 checks; fictional accounts in a loopback server; no physical phone.

[9] IndiCare report audit. *Current server processing benchmark*. `tools/benchmark_api.py` and `evidence/benchmark-current.json`, 26 October 2026. In process ASGI, temporary databases; raw samples and boundaries retained.

[10] IndiCare report audit. *Lost response retry comparison*. `tools/benchmark_idempotency.py` and `evidence/idempotency-before-after.json`, 26 October 2026. Initial `6757c0a:main.py` compared with current API using twenty matched synthetic pairs.

[11] IndiCare project. *Main branch version history*. Baseline and integration commits, 25–30 October 2026; Appendix D. `https://github.com/deepdevjose/Older-Monitoring-Wearable/commits/`.

[12] IndiCare team. *Contribution allocation supplied by José Manuel Cortes Ceron*. Project correspondence, 30 October 2026; structured record `evidence/team_contributions.json`; Appendix D.

[13] FastAPI. *Concurrency and async await*. Official documentation. `https://fastapi.tiangolo.com/async/`. Accessed 26 October 2026.

[14] Scikit learn. *Metrics and scoring quantifying the quality of predictions*. Version 1.8 documentation. `https://scikit-learn.org/1.8/modules/model_evaluation.html`. Accessed 26 October 2026.

[15] Android Developers. *Guide to app architecture*. Official documentation. `https://developer.android.com/topic/architecture`. Accessed 26 October 2026.

[16] Android Developers. *Make apps more accessible Views*. Official documentation. `https://developer.android.com/guide/topics/ui/accessibility/views/apps-views`. Accessed 26 October 2026.

[17] OWASP. *Password Storage Cheat Sheet*. `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`. Accessed 26 October 2026.

[18] SQLite. *Isolation in SQLite*. Official documentation. `https://www.sqlite.org/isolation.html`. Accessed 26 October 2026.

[19] IndiCare project. *Controlled ESP32 application upload and Network commissioning*. `evidence/current/FIRMWARE_NETWORK_EVIDENCE.json` and `FIRMWARE_DIAGNOSTIC.json`, 27 October 2026. App-only write, private backup, verified boot, Wi-Fi/NTP and aggregate sensor status; no physiological measurement study.

[20] IndiCare project. *Physical authenticated SPP protocol and normal application delivery*. `evidence/current/BT_PROTOCOL_EVIDENCE.json` and `APP_BLUETOOTH_SERVER_EVIDENCE.json`, 27 October 2026. 207 isolated protocol checks and a separate code 17 application to server observation; no caregiver notification acceptance.

[21] W3C Web Accessibility Initiative. *Developing Websites for Older People: How WCAG 2.0 Applies*. `https://www.w3.org/WAI/older-users/developing/`. Accessed 29 October 2026.

[22] Apple. *Human Interface Guidelines: Accessibility*. `https://developer.apple.com/design/human-interface-guidelines/accessibility`. Accessed 29 October 2026.

[23] Analog Devices / Maxim Integrated. *MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health*. Manufacturer datasheet Rev. 1. `https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf`. Accessed 30 October 2026.

[24] Melexis. *MLX90614 infrared thermometer: object and sensor temperatures, SMBus output and supply variants*. Manufacturer product documentation. `https://www.melexis.com/en/product/MLX90614/Factory-Calibrated-Infrared-Temperature-Sensor`. Accessed 30 October 2026.

[25] IndiCare project. *Controlled physical USB/SPP system run and delivery-quota improvement*. `evidence/current/DEVICE_SYSTEM_USB_EVIDENCE.json`, baseline/capture records in the same folder, `evidence/usb_performance.json` and `evidence/system_screenshots.json`, 30 October 2026. Isolated accounts/application ID; production Java/resources; physical Bluetooth and HTTPS. Reproduction: Appendix E.1.

A API contract and synthetic example

All paths below use `/api/v1`. A session is required except for registration and login. Exact validation and error bodies are defined by the source, not the historical screenshots.

Table 8. Selected current REST operations.

Method	Path	Purpose and access
POST	`/auth/register`, `/auth/login`	Create an educational account or start a session
GET/POST	`/me`, `/auth/logout`	Read own profile or revoke current session
POST	`/readings`	Adult uploads own source labelled snapshot; idempotent sample ID
GET	`/patients/{id}/overview`	Current reading and care state; own or approved resource
GET	`/patients/{id}/readings`	Persisted reading history; own or approved resource
GET	`/patients/{id}/metrics/{metric}/history`	Valid metric aggregates; period=day, week or month; source separated; own or approved resource
GET/POST	`/patients/{id}/learning/readings`	Saved synthetic predictions: adult adds own exercises; approved caregiver reads; thirty day retention
GET	`/patients/{id}/alerts`	Stored alert episodes; own or approved resource
POST	`/alerts/{id}/acknowledge`	Record the approved caregiver's own review
GET	`/notifications`, `/notifications/wait`	Private durable inbox or authenticated long poll
GET	`/ai/model`	Educational model identity and limitations
POST	`/ai/classify`	Synthetic features to class and confidence; no automatic reading or episode persistence

A.1 Observed inference request and selected response fields

The following normal example is retained in `evidence/benchmark-current.json`. Values are synthetic fixtures. Confidence is a model class score. The complete response also includes the model limitations.

```
POST /api/v1/ai/classify
Authorization: Bearer <session supplied at runtime>

{
  "source": "synthetic",
  "features": {
    "heart_rate_bpm": 75.0,
    "spo2_percent": 98.0,
    "body_temp_c": 36.5,
    "sample_age_seconds": 0.0
  }
}
```

```
{
  "classification": "Normal",
  "confidence": 0.999263,
  "model_id": "synthetic-health-v1",
  "source": "synthetic",
  "educational_only": true
}
```


Table 9. Expected failure behaviour of classification and data endpoints.

Condition	Behaviour to demonstrate
Missing or invalid session	Reject protected request; no patient data returned
Wrong patient relationship	Deny resource without revealing its contents
Missing feature, NaN, stale feature vector or device source	Classification rejected; no invented prediction
Same sample ID with changed content	Conflict response; existing sample preserved
Prediction limit exceeded	HTTP 429; user can retry later
Model unavailable or inference slots occupied	HTTP 503; no invented prediction is returned

B Reproduction installation and scenario evidence

B.1 Evidence package and word count

The prose count is generated by `tools/count_words.py`: 3,134 words in explicit `reportprose` environments. Headings and citation markers are excluded; including the short headings would remain within the permitted range. Tables, figures, title, references, test evidence and appendices are excluded as the assessment specifies. No source code listing appears in the main report.

Report evidence	Purpose
model-metadata.json	Training recipe, split, class metrics, artifact hashes and limitations
CURRENT_REVISION.tex	Current revision counts, APK identity and actual device scope
benchmark-current.json	26 October baseline timing samples and measurement boundaries
idempotency-before-after.json	Matched retry campaign with revision IDs and record counts
pytest-current.xml	26 October baseline 88 test rerun in JUnit format
jvm-current.txt	Baseline 2,830 assertion Java run
ANDROID_UI_EVIDENCE.json	Baseline 709 checks on SM-S948B; registration at text scale 2.0
hardware_photo.json	Original hardware photograph, source path, copy identity and scope
Report JSON snapshots	Dated code 13 build, baseline validation, integration and diagnostic records
current/VALIDATION.json	Current software checks with input hashes
current/ANDROID_HISTORY_UI_EVIDENCE.json	7,073 isolated UI assertions; 48 normal/enlarged-text captures
current/DEVICE_SYSTEM_USB_EVIDENCE.json	Measured real-account campaign, physical relay and native posting trials
current/DEVICE_SYSTEM_USB_CAPTURE_EVIDENCE.json	Later English capture pass; real API-backed views and retained record
usb_performance.json	Original before/after timing arrays, denominators and observation limits
Repository release/ records	Build identity and independently scoped installation, software and device evidence
crop_manifest.json	Source image hashes and exact crop coordinates

B.2 Build and isolated checks

Run the following from the repository root, with its Python environment installed. The benchmark scripts use temporary databases and fictional accounts, not the running server. They write separate rerun files rather than replacing the recorded results. The historic comparison requires Git commit 6757c0a; it adapts the legacy unversioned API and current versioned API to the same duplicate delivery campaign. It is not a throughput comparison.

```
python3 docs/report/tools/count_words.py
bash docs/report/build.sh
.venv/bin/python -m pytest tests
.venv/bin/python mobile/scripts/test_reading_summary.py --jdk /usr
.venv/bin/python docs/report/tools/benchmark_api.py
.venv/bin/python docs/report/tools/benchmark_idempotency.py
```

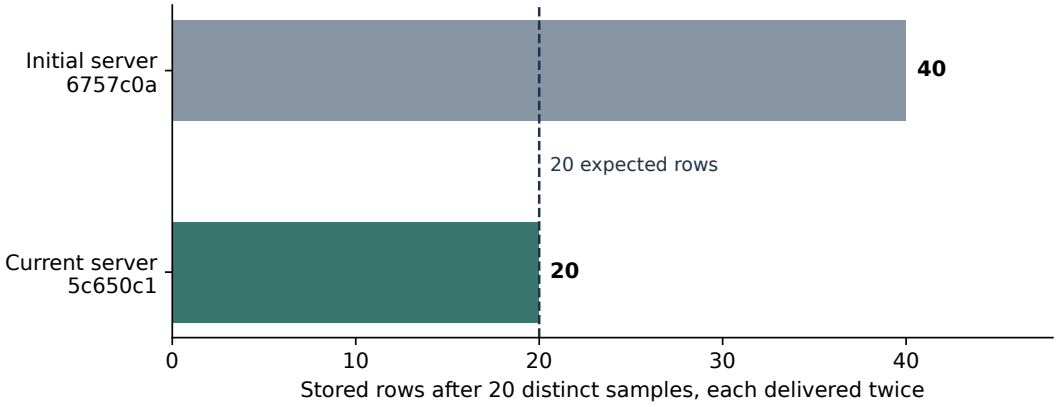


Figure 9. Measured chronological improvement in backend persistence. Forty successful responses represent twenty unique samples delivered twice. The baseline creates twenty extra rows; the current API preserves twenty unique records. The comparison excludes handset and network behaviour.

B.3 Installation and execution procedure

The current APK and source archive are distributed under `release/`. The application is version 1.0.0, code 20, minimum API 24 and target API 37. Its SHA-256 is `7aa2bdd5cd97512057389dcd3586305f6e57678801f1ca511c795c1b2d933f7e`. Device execution was recorded as follows: Code 20 installed on S22 / Android 13 with verified APK identity and readonly HTTPS client check. Separate campaigns: 7,073 isolated preview assertions with 48 captures at 100%/200% text; 119 actual-system assertions using production Java/resources in an isolated package. Main-app onboarding was safely skipped to preserve an existing session; the earlier 878 checks belong to code 19. The older report snapshots retain code 13's separate installation and 709-check campaign. The operational guides are `docs/SERVER.md` and `docs/PUESTA_EN_MARCHA.md`; use the exact endpoint and device identity for the selected transport.

1. Prepare the Python environment from `requirements.txt`; initialise the local server using `./carectl init -host YOUR_LAN_IP`. Keep generated keys private.
2. Use `./carectl start` and `./carectl status` to check HTTPS service readiness. Ensure the public CA bundled in the APK matches that server. Follow `docs/SERVER.md` when regenerating a build.
3. Install `release/CareMonitor-1.0.0.apk` using Android's authorised local installation flow and retain its hash and device identity. The installation, preview and isolated system campaigns used S22 Android 13/API 33. Preview fixtures do not grant permissions or activate sensors; the separate system campaign exercises actual acquisition and HTTPS.
4. Use fictional adult and caregiver accounts for server experiments. Registration presents role, name, email and password as separate steps; the adult issues an invitation and approves the caregiver request. The server enforces that relationship.
5. Run `./carectl demo` with fictional adult credentials. This sends three synthetic rule samples; verify one episode and independent caregiver review. Rule episodes are separate from saved model exercises.
6. Tap a metric card and select Day, Week or Month. Inspect the range, chart, units, gaps and coverage statement. In caregiver mode, select an approved adult and check the same authorised history. A missing current reading must not hide retained history.
7. Open Settings, then Learning insights. Under a fictional adult account choose each synthetic CSV example, inspect the three features, then classify and save. Verify class, score and timestamp in Recent exercises. The approved caregiver can read the same history but cannot submit for the adult. The original `/api/v1/ai/classify` remains a separate stateless API.
8. Reproduce interface checks only on an isolated test device or preview application, selecting the exact serial and preserving real account state. The onboarding instrumentation uses fictional in-memory profiles and has an active-session guard. Do not clear a user's app data to satisfy that guard. Current fixture evidence is scoped separately from account, backend and Bluetooth operation.

B.4 Application latency and improvement protocol

Select an initial revision and an improved revision before the experiment. Use the same device, dataset fixture IDs, server hardware, network and permissions. After identical warmup, record at least thirty trials per scenario. Measure API time from dispatch to parsed response, and caregiver latency from the first qualifying synthetic input to visible notification. Keep raw timestamps, request IDs, failed trials and timeouts. Report N, mean, p50 and p95 for both revisions; do not silently omit failed deliveries.

For missed Critical alerts, divide Critical scenarios without a required visible alert within the declared observation window by all Critical scenarios that should generate one. For false Warnings, divide Normal scenarios producing an unwanted Warning by all Normal scenarios. Define the observation window before testing. Model predictions and rule alerts have different semantics; a latency experiment must identify the rule output and its observation window. Separately report duplicates and data loss after reconnection.

B.5 Scenario coverage matrix

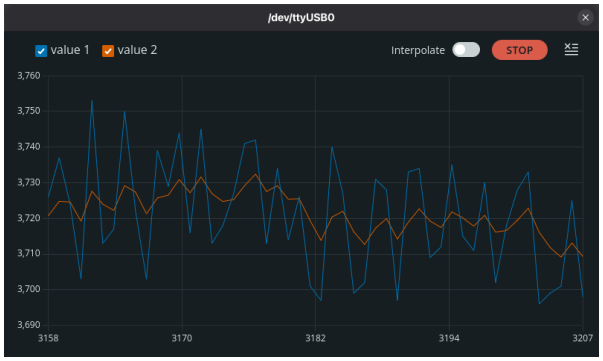
Table 10. Reproducible scenarios, observed evidence and execution boundaries.

Scenario	Input and expected result	Recorded evidence and scope
Normal	Fixed synthetic normal features; return Normal and score; valid normal snapshots create no rule episode	Model/API tests, preview fixture and saved real-API learning examples; no clinical result inferred
Deteriorating	Move from eligible normal snapshots to three distinct abnormal snapshots; one episode after persistence	Sustained condition and recovery test in isolated server fixtures
Critical	Select known Critical CSV fixture; return expected class; independently exercise an abnormal rule sequence	Held out model matrices and server episode campaign; delivery rate is a separate measure
Missing data	Omit required model feature; reject. Missing optical contact must not trigger or recover a rule	Missing feature and invalid contact tests; care views retain explicit unknown states
Invalid data	NaN, unexpected field, unsupported feature or stale age; reject classification	`test_ml.py`, `test_ai_api.py`, request limits tests
Network failure	Lose upload response and retry same ID; retain one row and active episode	Matched backend comparison and relay policies; excludes physical radio interruption
Unauthorised	Query unlinked adult, then revoke previously approved caregiver; disclose no resource	Server IDOR and revocation tests; native caregiver wearable controls are also rejected
Historical view	Mixed valid and invalid observations over Day/Week/Month; correct weighted mean and explicit gaps	`tests/test_history.py`; aggregation and retention; native real-account surface history under both roles
Saved confidence	Add synthetic exercise, retry same identifier, then read from approved caregiver account	`tests/test_learning.py`; conflict/access tests; real-API saved class/confidence reviewed under both roles
Recovery	Three eligible observations no longer meeting the rule; recover episode; later new sustained event may create another	Sustained condition recovery test; review is not recovery

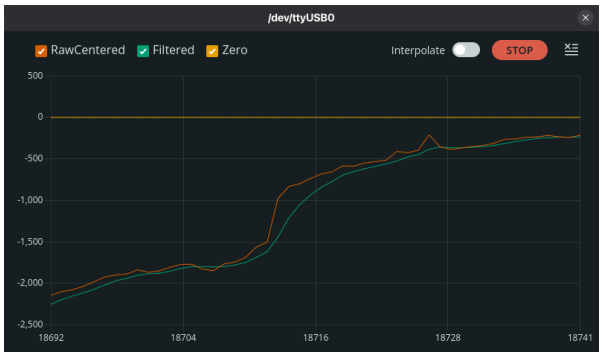
Exact automated anchors in `tests/test_api.py` include `test_sustained_condition_one_episode_valid_recovery_then_new_episode`, `test_idempotency_conflict_and_capture_order`, `test_idor_cannot_read_or_write_other_patients` and `test_revoke_removes_data_and_queued_notifications`. Model input boundaries are exercised by `tests/test_ai_api.py::test_synthetic_api_rejects_device_and_missing_features`. These names locate executable checks; they do not represent installed Android tests.

C Historical development images

The following supplied captures are cropped for relevance. No output or values have been synthesised. Source paths and crop coordinates are recorded in `evidence/crop_manifest.json`. They document historical software experiments, separately from the assembly in Figure 2 and the Android code 13 interface campaign.



(a) Raw and smoothed ADC traces.



(b) Centred input, filtered output and zero reference.

Figure 10. Serial plotting during development on 13 October 2026. The uncalibrated ADC plots support qualitative signal conditioning observations. Axes do not establish physical time, clinical accuracy or a quantified before and after noise reduction.



Figure 11. Local API availability and successful historical uploads on 13 October 2026. The old `/sensor-data` route returns HTTP 200 while the optical module reports `no_finger`. Successful transport does not establish measurement validity, persistent storage or the final versioned API.

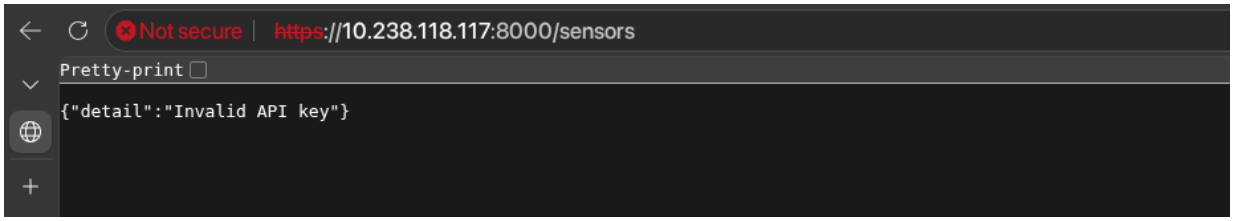


Figure 12. Historical authentication rejection on 15 October 2026. The browser reports an invalid API key and also marks the connection insecure. This establishes a negative authentication path for that version; it is not evidence of trusted HTTPS or the current account based contract.

D Project team and development history

D.1 Individual and shared contributions

Table 11 records the task allocation supplied by José Manuel Cortes Ceron on 30 October 2026. Associated artefacts locate the deliverables; `evidence/team_contributions.json` retains the declaration.[12]

Table 11. Team contribution matrix based on the supplied allocation.

Team member	Confirmed contribution	Associated artefacts
Nathan Ratichon 2362789	Participated in exploratory fall-related trials intended to support model development; tested individual sensors with Cheng; helped write the report progressively, recording the development steps.	Sensor acquisition in `esp32.ino`; historical captures in Appendix C; submitted draft and report.
Guannan Ge 2364229	Developed the app UI/UX with Cheng; developed the first combined-sensor firmware with José, reading sensors together and transmitting JSON.	Android views and role flows; integrated firmware and JSON envelope; Figures 3 and 1.
Jose Manuel Cortes Ceron 2686162	Developed the backend; built the initial integrated firmware with Guannan; refined JSON and sensor selection, removing the light sensor and microphone to simplify acquisition and transmission.	API, persistence and security modules; firmware `printJSON()`; current three-sensor wiring in Figure 19.
Cheng Zhang 2361292	Developed the app UI/UX with Guannan; tested the individual sensors with Nathan before their combined integration.	Android interface and accessibility flows; individual acquisition routines; historical sensor captures.

All four members developed the idea, selected the sensors and their purpose, and defined the architecture, data exchange and protocols. The requirements, sensor rationale and system atlas represent this shared design. The tutor is Matilda Isaac; `metadata.tex` maintains names and IDs.

The exploratory fall trials are separate from the reported model evaluation; no performance metric is attributed to them. The SisFall experiment uses public records (`datasets/README.md`). Sensor and JSON simplification is credited without asserting an unmeasured latency or payload-size improvement.

D.2 Versioned development record

Date	Commit	Recorded subject
25 Oct 2026	`27a72d4`	Initial commit
25 Oct 2026	`6757c0a`	upload files
25 Oct 2026	`7e6c494`	upload csv files and data for ML part
26 Oct 2026	`5c650c1`	update: mayor fixes
30 Oct 2026	`5d86dfb`	Finalize system architecture report and code 20 device evidence
30 Oct 2026	`b7dcb80`	Keep generated and historical system event copy in English

Git records one identity, José Manuel Cortes Cerón. The first four commits form the submitted draft's baseline; the final two record code 20 evidence and English system events. Build/campaign hashes identify the artefacts exercised before later editorial changes. This chronology records integration; the supplied declaration establishes task attribution.[11]

E Demonstration evidence and evaluation boundaries

The completed evaluation combines native interface checks, synthetic model/rule experiments and an integrated physical bench campaign. Their records retain separate versions and execution boundaries rather than treating every test as a single deployment.

The installed production code 20 has a verified APK identity and readonly HTTPS client check; its onboarding campaign was safely skipped because an account already existed. A separate preview ran 7,073 assertions and captured 48 views at normal/enlarged text. The earlier 709-check onboarding result retains code 13's identity.

The 30 October system campaign used production Java/resources in an isolated application ID with controlled accounts, real Bluetooth SPP and HTTPS. It verified caregiver request and adult approval, retained 76 physical readings, reached zero pending uploads and rendered both roles from actual API responses. Learning insights saved Normal, Warning and Critical synthetic exercises. Final notification trials posted 30/30 native caregiver notices within ten seconds after the bounded-window correction. The following subsections retain the observation chain, before/after timing and role-separated captures.[25]

To reproduce the integrated path:

1. Follow Appendix B to prepare the trusted local server and exact APK/endpoint. Record device, build and transport identities before the run.
2. Use controlled adult/caregiver accounts. Connect the adult's wearable, then request and approve the care relationship; verify retained readings and the upload queue before opening caregiver views.
3. Tap a metric card to fetch retained history, and separately classify/save the synthetic learning examples. Capture the selected person's native views with the corresponding server record and source identity.
4. Repeat notification trials using the declared screen state, observation window and permissions. Retain every successful post and timeout; do not substitute model accuracy or server-only timings for delivery results.

Table 12. Evaluation domains and the evidence supporting each claim.

Domain	Recorded evidence and interpretation	Assessment relationship
Android interface	Installed code 20: identity and readonly HTTPS check; 7,073 isolated preview assertions; 119 actual-system assertions in an isolated package. Main-app onboarding skipped to preserve the existing session; historical code 13 separately contributes 709 onboarding checks	Mobile functionality and programming
Classification	Two trained models, held out matrices, authenticated serving and native real-account display of saved synthetic scores	AI, REST and mobile integration
Persistence and alerts	229/229 tests, 24/24 checks, plus dated twenty retry pairs and twenty episode trials	Data integration and testing
Hardware assembly	Original photograph and source-derived wiring; the following physical campaign records 76 retained device readings	Architecture and demonstration
Development trace	Team-confirmed contribution matrix, associated artefacts, dated commits and build/campaign input hashes	Configuration and documentation

MainActivity presents readings, history, alerts and Settings. LearningActivity presents saved synthetic classes and scores with role-enforced access. The following campaign adds physical acquisition, persistence and native notification posting within its stated USB and screen-on boundaries. None establishes clinical efficacy, sensor accuracy or an audible emergency response.

E.1 Observed USB system execution: 30 October 2026

The current bench campaign uses real Bluetooth Classic SPP acquisition and real HTTPS requests from production Java and resources in an isolated Android application ID, `org.caremonitor.reportqa`. Its permissions and services come from the production manifest. A separate SQLite deployment on PC loopback port 19444 uses the existing trusted CA; adb reverse carries its HTTPS connection. Production accounts, the main application data and the laptop's Wi-Fi/VPN are preserved. The main installed application's separate USB deployment remains at port 19443.

The campaign creates controlled adult and caregiver accounts with fictional names, verifies that an unapproved caregiver sees no adult, requests a link and approves it, receives the physical wearable stream and checks the upload queue against retained server records. It then renders the native adult/caregiver pages using actual HTTPS results. These captures are not in-memory preview fixtures. Saved learning exercises explicitly use synthetic CSV examples. The temporary package is removed at the end, including its credentials and installation key; no default credentials are distributed.[25]

Table 13. Recorded end-to-end observation boundaries. The first sample checkpoint precedes the later capture and database checkpoints, so their cumulative counts differ.

Observation	Evidence and meaning
Phone and software	Samsung S22 SM-S901E / Android 13, code 20 production Java/resources in the isolated report package. Its full input hash map is retained; the production APK installation has a separate hash and readonly client check.
Physical transport	67 received samples at the first checkpoint in the measured campaign; both GY906 and MPU6050 reported online. The server later retained 76 device readings and the outbox reached zero pending.
Optical validity	Contact was supplied for a bench check. Reported optical status and validity flags are retained together: warming/invalid states are not presented as valid physiological readings. No accuracy or clinical population study is inferred.
Installed firmware	The installed image still reports legacy microphone/light fields. These do not establish connected modules. The corrected three-sensor source compiled without being flashed.
Role and AI execution	Approved caregiver reads the adult's retained history. Three synthetic learning exercises are saved through the real API and reviewed under both roles. Device temperature is not sent to the body-temperature model.
Capture campaign	76 device readings retained; representative record 300 with stable sample ID below. The device screen uses UTC+8 and may show 31 October while the UTC log remains 30 October.

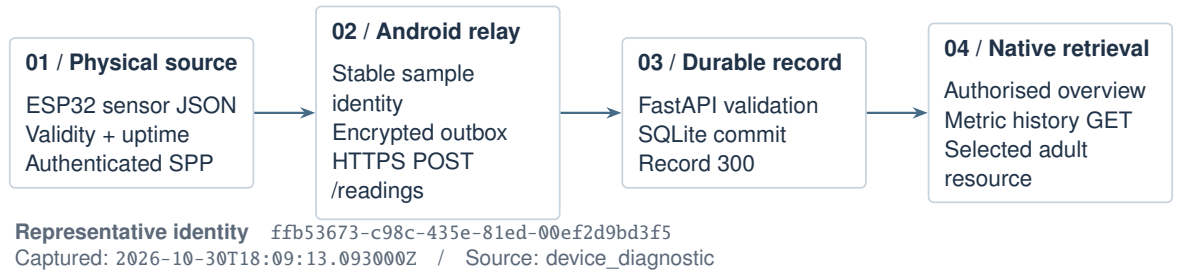


Figure 13. Traceable sensor-to-API-to-mobile chain. Stable identity and commit/queue confirmation establish persistence; the native chart then reads retained aggregates for that account and interval. A short bench interval does not represent a day-long measurement trace.

E.2 Measured mobile response and native notification delivery

Thirty S22 overview requests are timed from Android dispatch through parsed HTTPS response. Notification trials start at dispatch of a controlled synthetic confirmed SOS and end when NotificationManager exposes a

new native care notification for the approved caregiver. Trials are seven seconds apart; each has a ten-second observation window. The two roles share one physical handset sequentially; the caregiver's foreground connection is active during delivery measurement. This is not a two-handset, locked-screen, audible-alarm or human-response experiment.

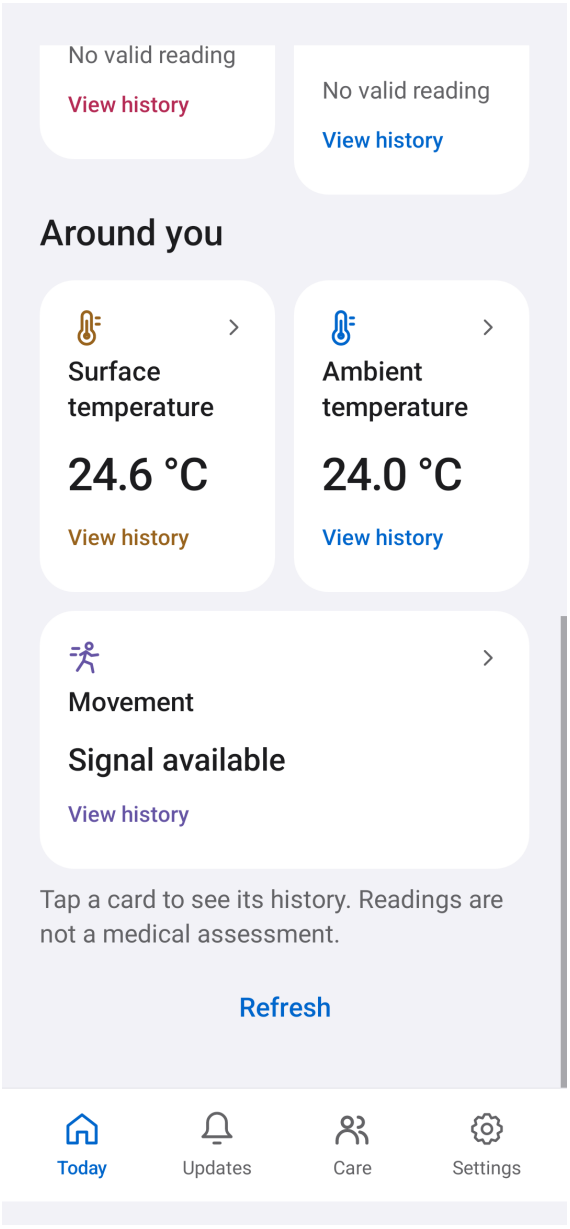
Table 14. S22 USB measurements, in milliseconds, using linear-interpolated percentiles. Notification latency is conditional on successful posting; failed trials remain in the delivery denominator.

Operation / revision	N	Mean	p50	p95
Overview HTTPS round trip, before	30	57.80	71.19	75.69
Overview HTTPS round trip, final	30	58.11	71.08	77.04
Native posting, before: 21/30; 9 timeouts	21	546.51	559.35	977.38
Native posting, final: 30/30; 0 timeouts	30	636.36	662.45	1094.37

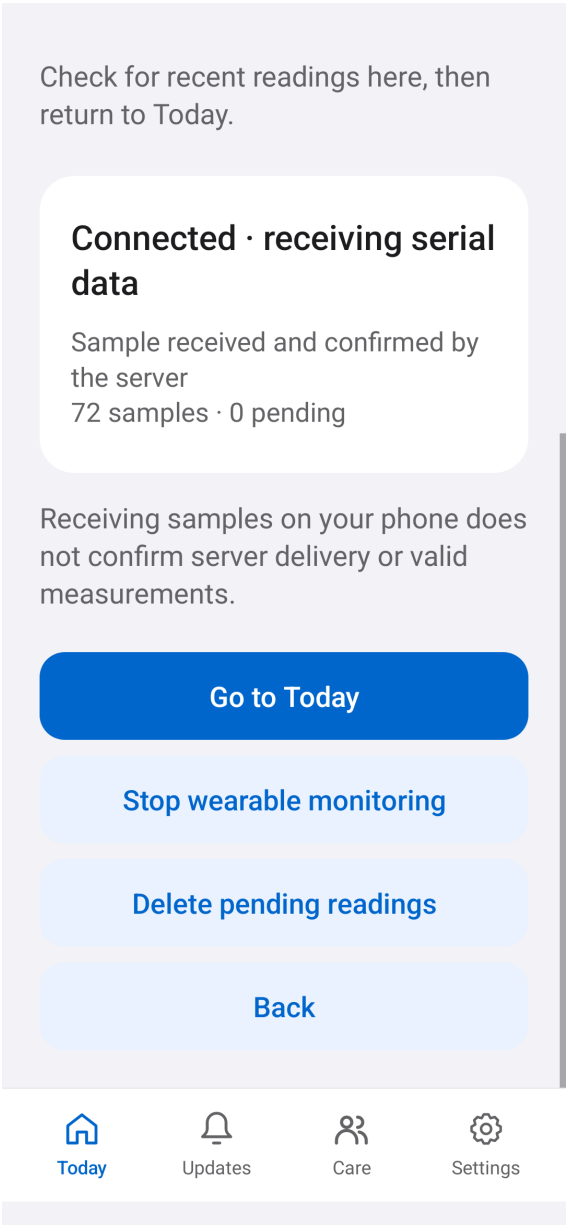
The first campaign exposed the S22's 25-notification package limit: Android rejected further native cards after the tray accumulated earlier cards and connection notifications. The original smoke runner completed but its nine recorded timeouts fail the strict delivery criterion; they have not been removed from the baseline. NotificationWindow now bounds event cards to twelve, including those scheduled earlier in the same batch. Evicting a native card neither acknowledges nor deletes its durable inbox item or alert. Two subsequent thirty-trial campaigns posted every requested notification within the observation window. The improvement is delivery completeness under this quota condition, not reduced successful-posting latency.[25]

The final source also translates newly generated and recognised historical system event defaults into English without changing user-authored text or rewriting encrypted historical records. A later capture pass records that presentation correction. Original timing arrays retain their own run identity and are not relabelled as newly measured values.

E.3 Older adult: live acquisition and understandable readings



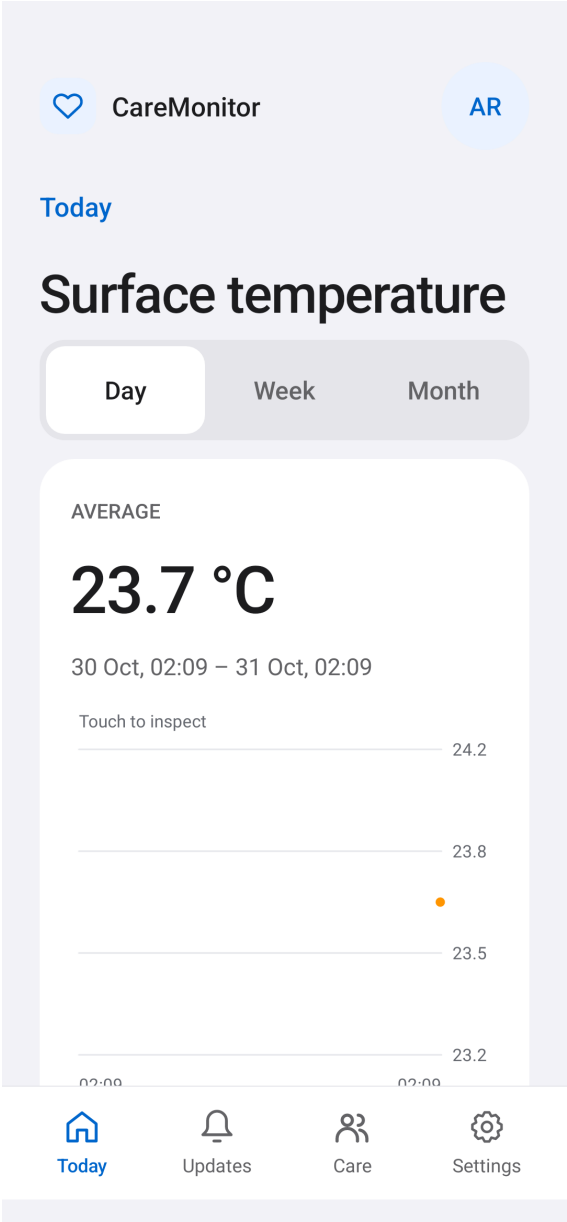
(a) Current thermal and movement cards; retired light card absent.



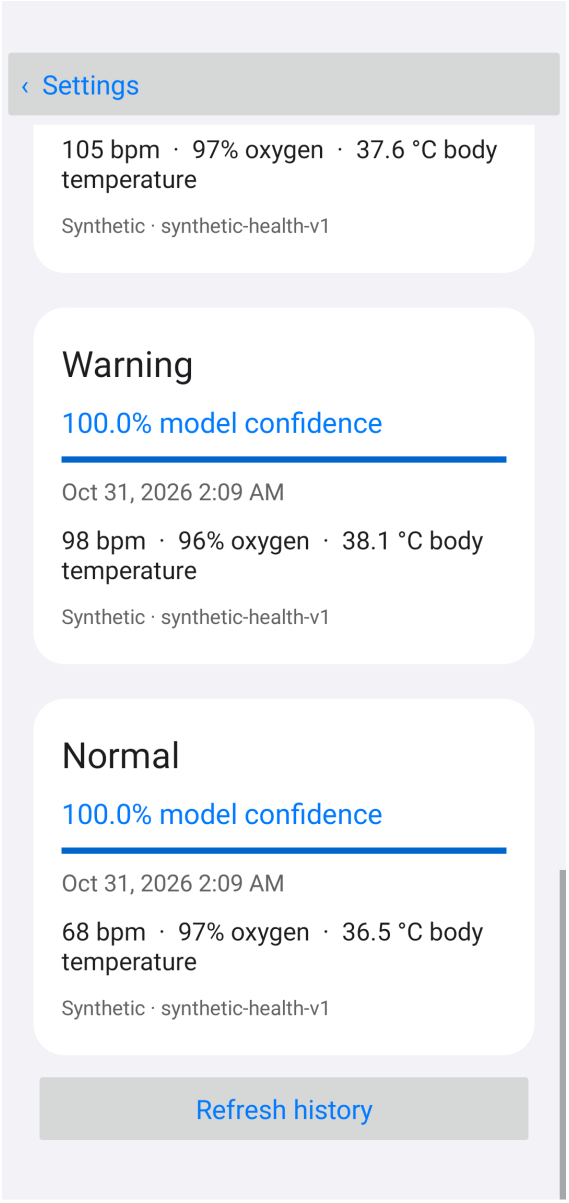
(b) Authenticated reception, server confirmation and zero pending uploads.

Figure 14. Native S22 adult views during the real USB/SPP capture run. These controlled-account screens retain sensor validity and show live relay state. Counts continue increasing while pages are captured; they are checkpoints rather than a fixed total. The capture's complete hashes and original filenames are retained.

E.4 Older adult: stored history and separate learning results



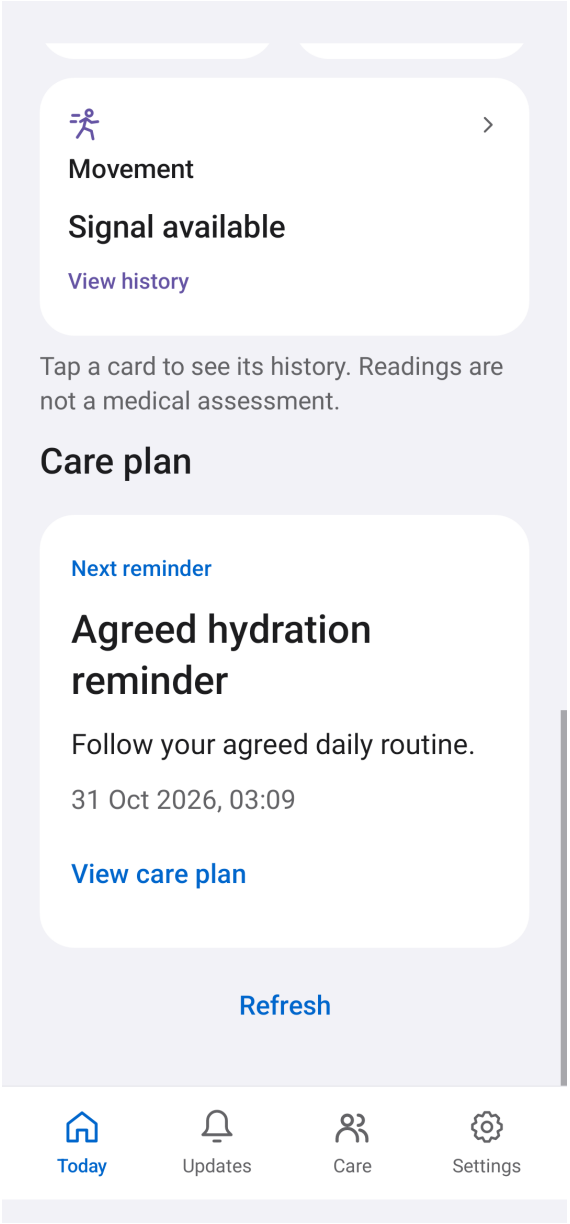
(a) Surface history from retained device observations.



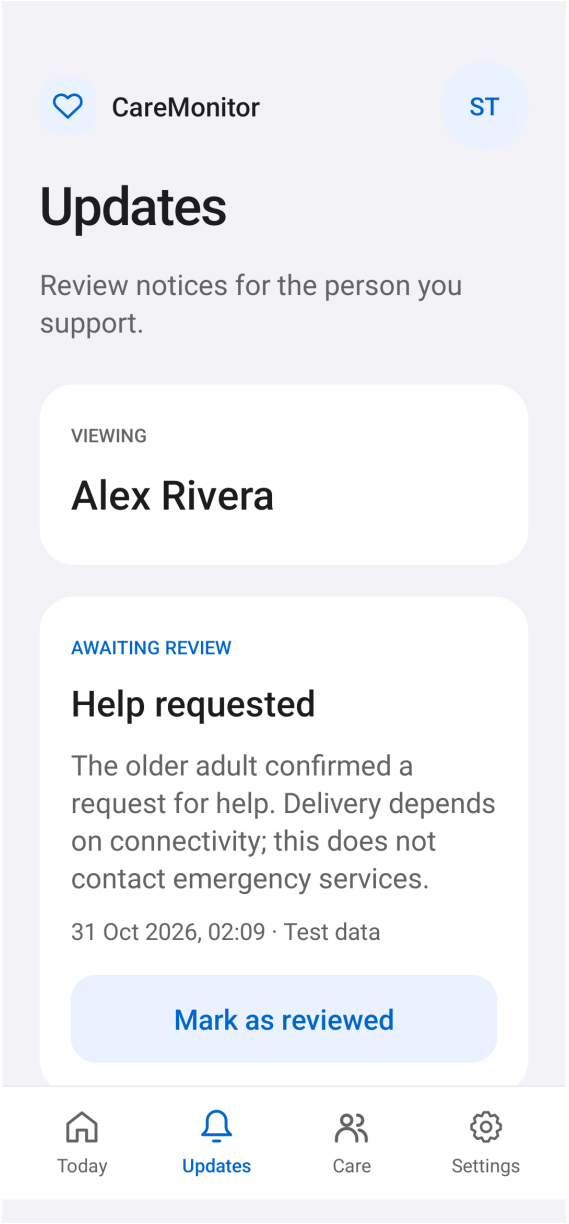
(b) Saved synthetic classifications and model scores.

Figure 15. Adult history and learning exercise results from real HTTPS responses. The short bench run contributes a small number of aggregate chart points. The synthetic confidence display is separate from the physical surface-temperature history; its score is not medical certainty.

E.5 Caregiver: selected person and reviewable notices



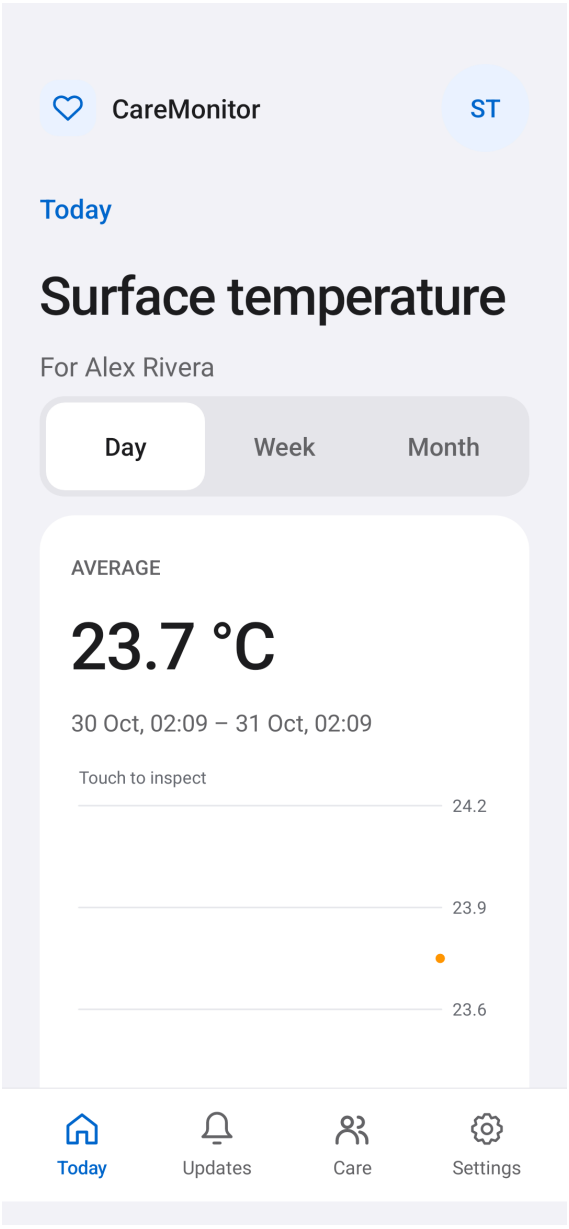
(a) Shared readings and the agreed care reminder.



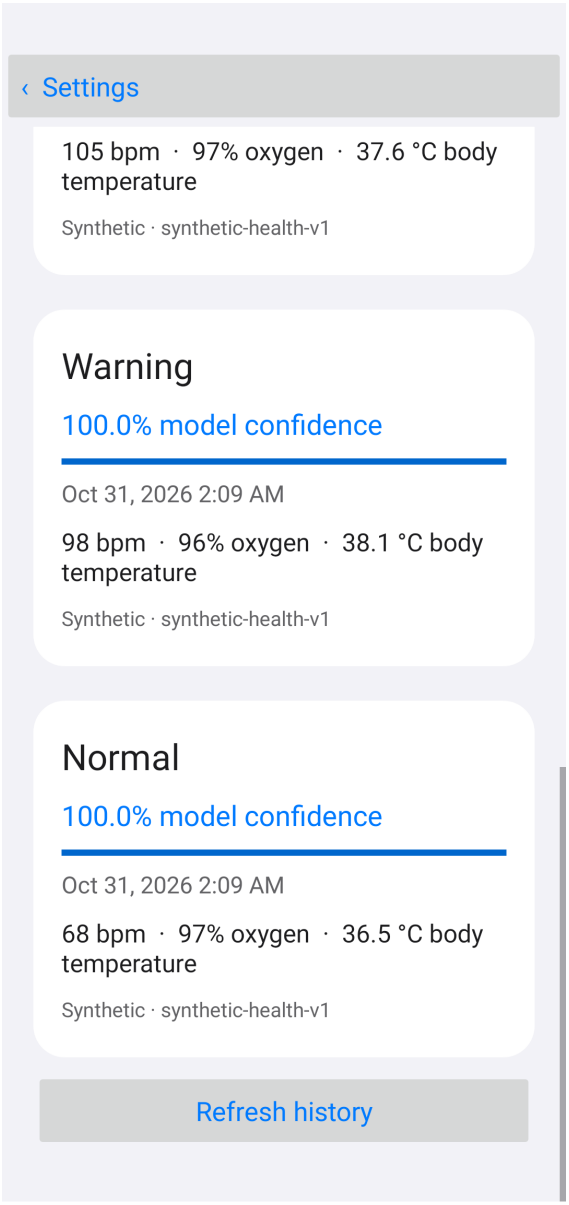
(b) Controlled help notice awaiting this caregiver's review.

Figure 16. Native approved-caregiver views from the isolated capture campaign. The help event is labelled test data, and acknowledgement is a human review action rather than proof that an emergency response occurred. Names identify fictional test accounts; the device observations originate from the bench assembly.

E.6 Caregiver: authorised history and saved synthetic confidence



(a) History explicitly names the selected adult.



(b) Read-only access to that adult's retained exercises.

Figure 17. Caregiver access is acquired through an invitation, request and adult approval before these real API-backed views are captured. Historical charts and saved class/confidence are demonstrated on the handset, while location/audio permissions remain separate opt-ins.

F Current system architecture atlas

This atlas documents the user-specified three-sensor wearable and the canonical Android/FastAPI implementation. The removed microphone and light module are absent from the current wiring topology. Legacy JSON fields and a BLE transport client may remain compatible software contracts; they do not identify additional installed hardware. The earlier assembly photograph is retained with its date rather than altered to resemble a new photograph.

Reading the figures. Blue and teal group technical responsibilities; colour does not assert validation. Every diagram names its arrows: runtime transfer, dependency, reply, inheritance or electrical net. The captions distinguish source-derived structure, executed campaigns and future capability.

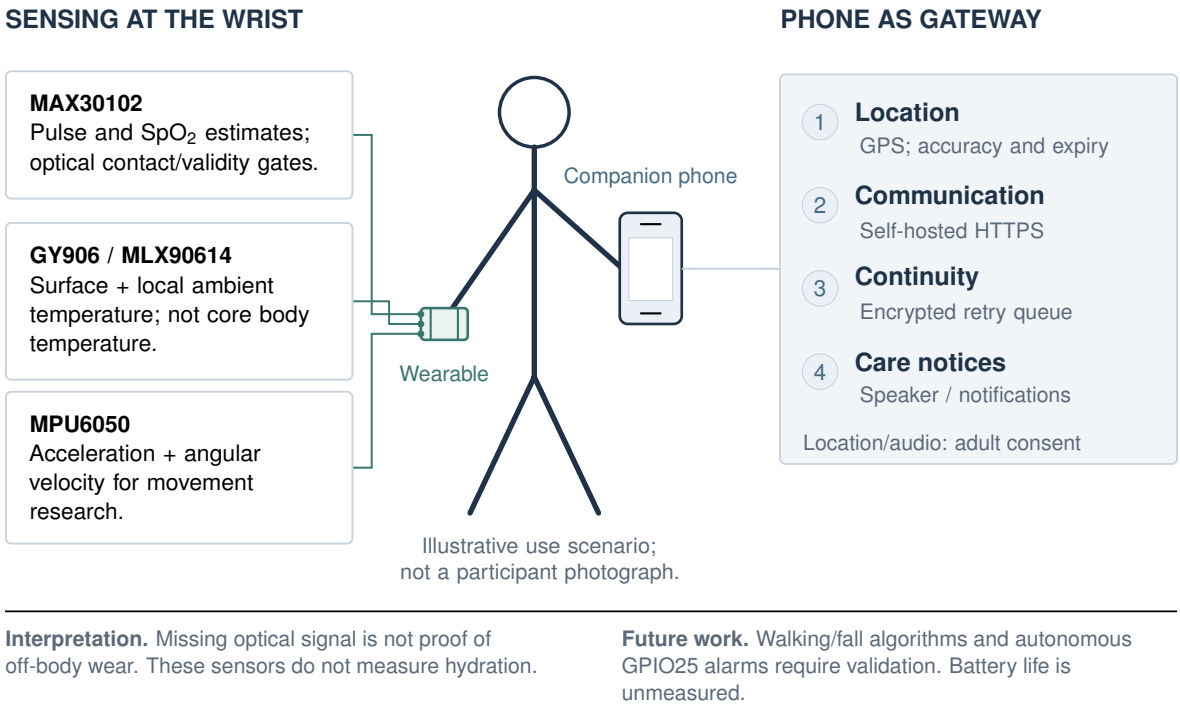
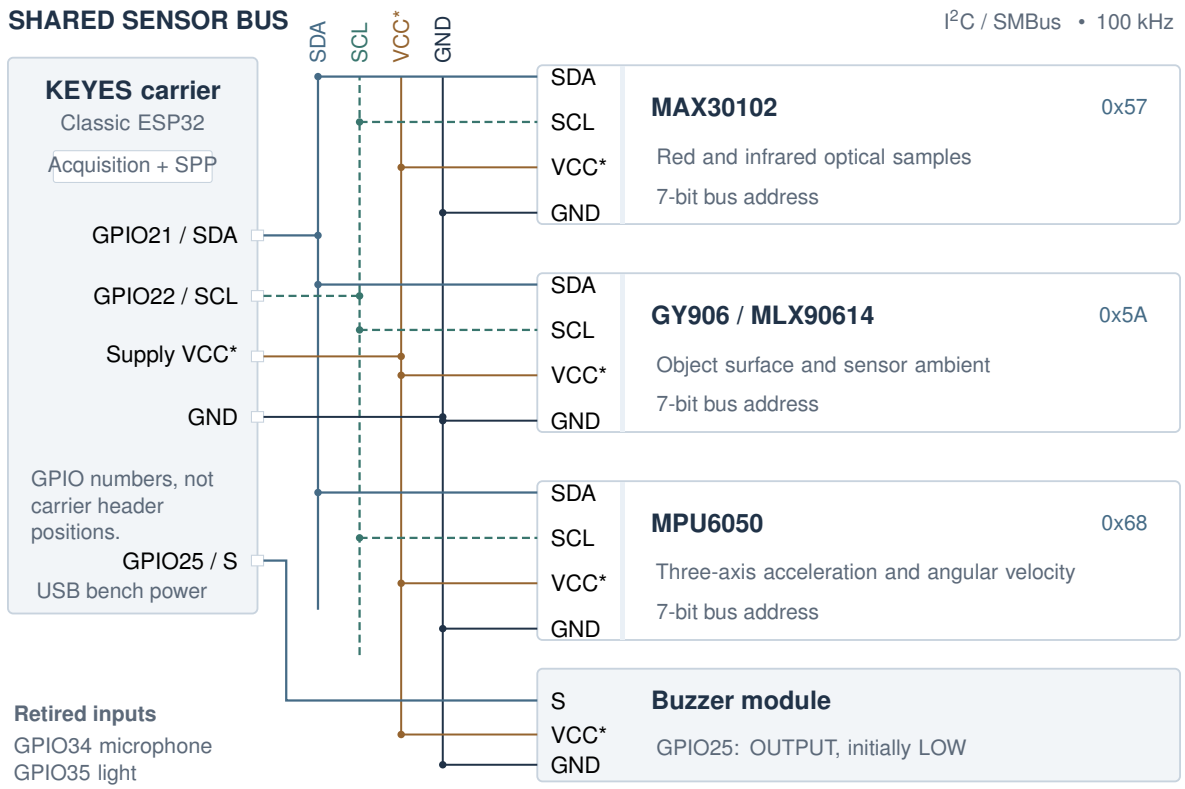


Figure 18. Component roles in the wearable–phone design. The phone supplies location, networking, storage and sound; the wearable supplies optical, thermal and motion observations. Physical readings are not treated as diagnoses.

F.1 Electrical connections and pin ownership



Electrical convention: dots join nets; unmarked crossings do not. INT, AD0 and auxiliary IMU signals are not read by this source.

VCC* is breakout-dependent. Use a common ground and ESP32-compatible 3.3 V logic. Confirm the actual breakout regulator, pull-ups and carrier rail before wiring; the bare MAX30102 has separate core/LED supplies. Confirm the buzzer's supply and driver/current requirements.

Figure 19. Source-derived functional wiring for the current three-sensor wearable. This is a GPIO and bus map, not a certified KEYES header pinout or a cable-by-cable inspection of the older photograph. The corrected source compiled; it was not flashed during this evaluation.

F.2 Transport protocols and wire formats

Table 15. Implemented communication boundaries. Cryptographic controls and local persistence are distinguished from transport protocols.

Boundary	Protocol / format	Contract and protection
Sensors / ESP32	I^2C / SMBus, 100 kHz	GPIO21 SDA, GPIO22 SCL; 7-bit addresses 0x57 / 0x5A / 0x68. Common ground and compatible logic.
ESP32 / phone	Classic Bluetooth SPP; RFCOMM UUID 0x1101; UTF-8 NDJSON	Encrypted link plus CM1 HMAC-SHA256 proofs. One active phone; telemetry after CM1 READY. This firmware does not expose BLE GATT.
Phone / PC carrier	USB adb reverse, or LAN IP/TCP	USB maps phone loopback to PC loopback. LAN normally uses port 8443. Both carry the same certificate-verified HTTPS application traffic.
App / FastAPI	HTTPS REST JSON, /api/v1	Certificate chain and hostname checks; Bearer token; per-resource role and approved-link authorisation.
Private care inbox	Authenticated HTTPS long polling	Incremental durable inbox IDs. Delivery acknowledgement, human review and episode recovery are separate operations.
Services / SQLite	sqlite3 transactions and WAL	Unique sample IDs, atomic fanout and encrypted sensitive fields. AES-GCM is field protection, not full-file database encryption.
Optional board clock	Wi-Fi / IP / UDP / NTP	Time synchronisation requires reachability. Missing wearable timestamps use phone receipt time; NTP does not authenticate measurements.

F.3 Actors and approval boundaries

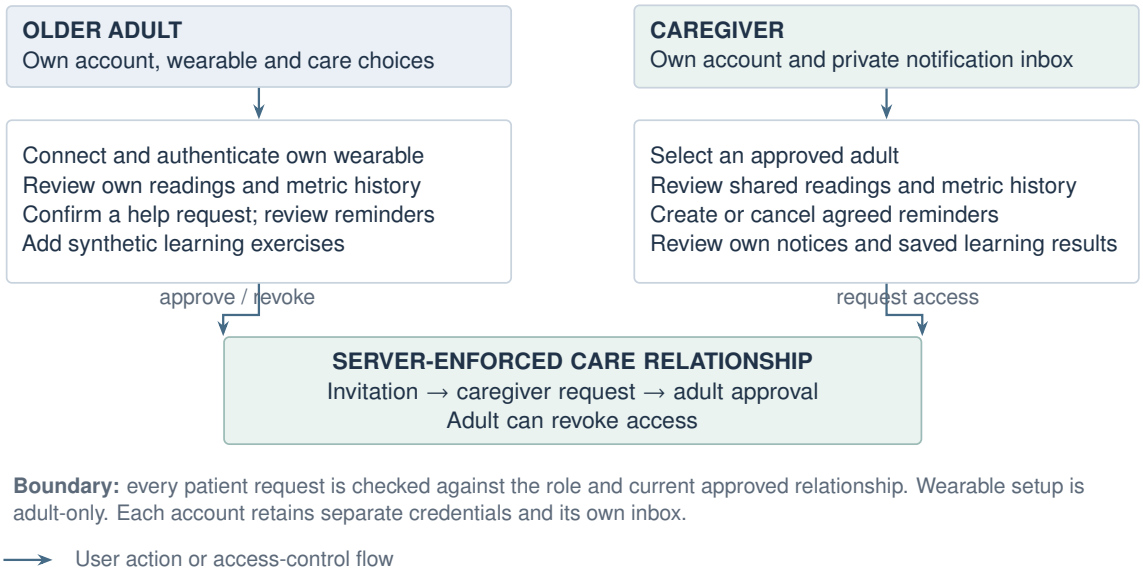


Figure 20. Implemented adult and caregiver use cases. An invitation alone grants no access; approval and revocation are checked by FastAPI for each patient resource.

F.4 Mobile navigation and task flow

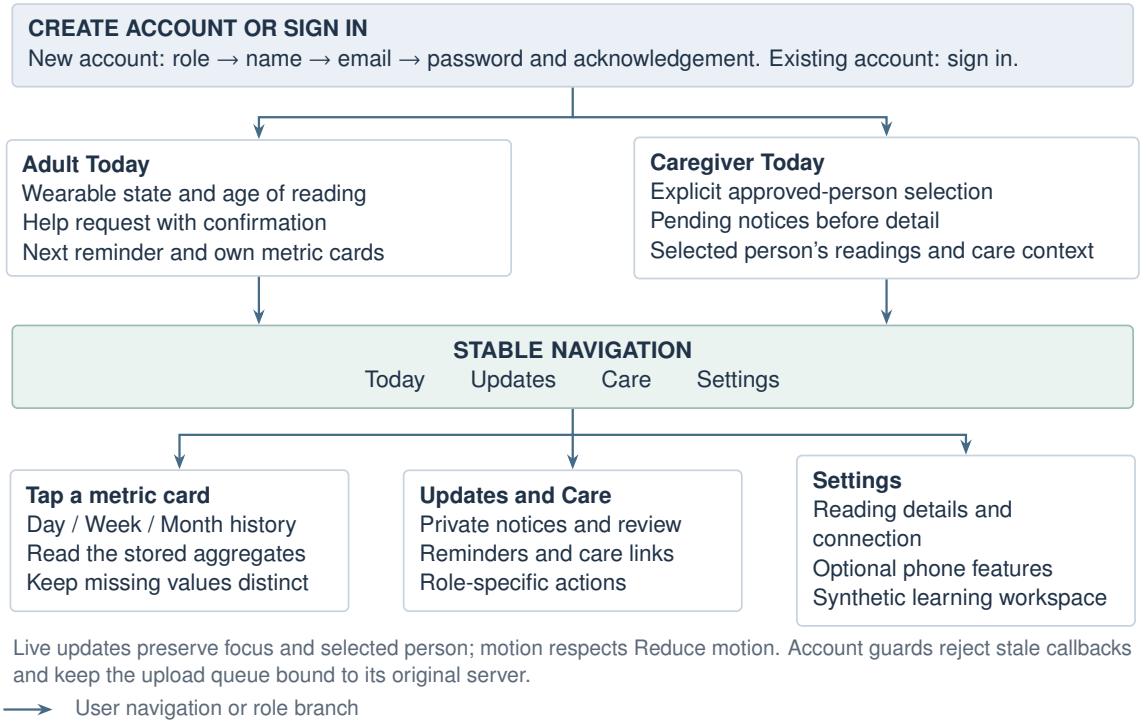


Figure 21. Native mobile workflow and information hierarchy. Frequent care tasks are visible on Today; administration and detailed readings are reached through Settings. Learning is a synthetic exercise workspace, separate from physical wearable interpretation.

F.5 Selected production classes

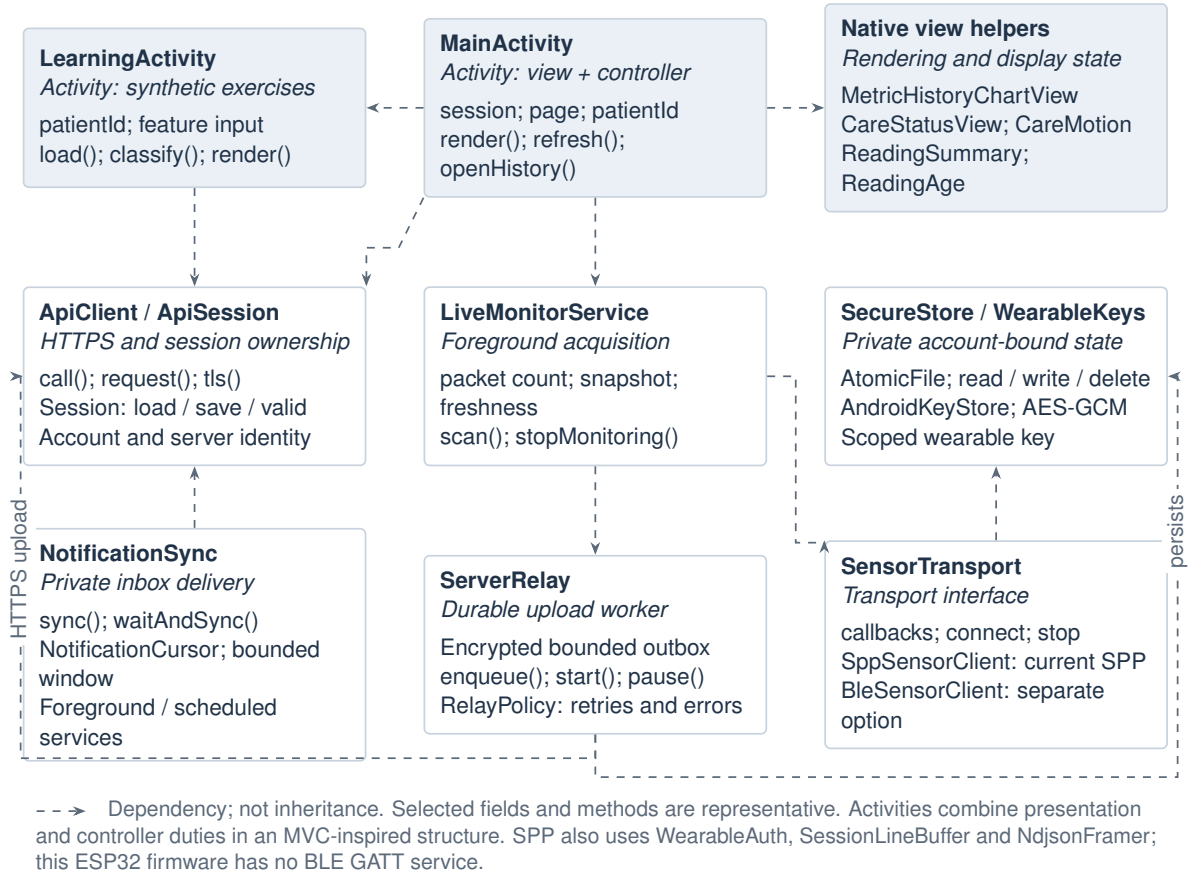


Figure 22. Source-derived Android responsibilities and dependencies. The reusable transport, persistence, notification and API helpers support the activities; the diagram does not imply a complete MVVM architecture.

F.6 Permissions and optional phone capabilities

Table 16. Feature-to-permission map from the manifest and runtime checks. Optional phone sensing is separate from the removed wearable microphone.

Action	Android capability	Scope and lifetime
Sign in; upload; read	INTERNET ACCESS_NETWORK_STATE ACCESS_LOCAL_NETWORK (API37+)	Certificate and host verification also apply over USB. The S22/API33 has no API37 local-network prompt.
Find and connect wearable	BLUETOOTH_SCAN BLUETOOTH_CONNECT (API31+) Legacy permissions through API30	Requested during adult-only wearable setup. Scanning on older Android requires location permission.
Keep wearable connected	FOREGROUND_SERVICE; connectedDevice type and its API34+ permission	Explicit connection action, visible foreground status and Stop action. Connection is not autonomous fall detection.
Enable private notices	POST_NOTIFICATIONS on API33+; specialUse foreground-service type on API34+	Explicit opt-in to NotificationChannelService; HTTPS long polling and visible connection status. No third-party push.
Share phone location	ACCESS_COARSE_LOCATION ACCESS_FINE_LOCATION Location foreground-service type	Adult's server consent plus runtime grant. Accuracy and expiry are retained; sharing can stop and consent can be revoked. No background-location permission.
Enable sound checks	RECORD_AUDIO; microphone foreground-service type	Separate adult consent and visible service. Human confirmation is required; raw audio stays on the phone. Loudness alone does not establish a fall.
Play a notice sound	Notification channel and system volume	No microphone permission. Channel settings, volume, Do Not Disturb and OS restrictions affect audibility. Native posting does not prove the user heard it.

F.7 Server components and source separation

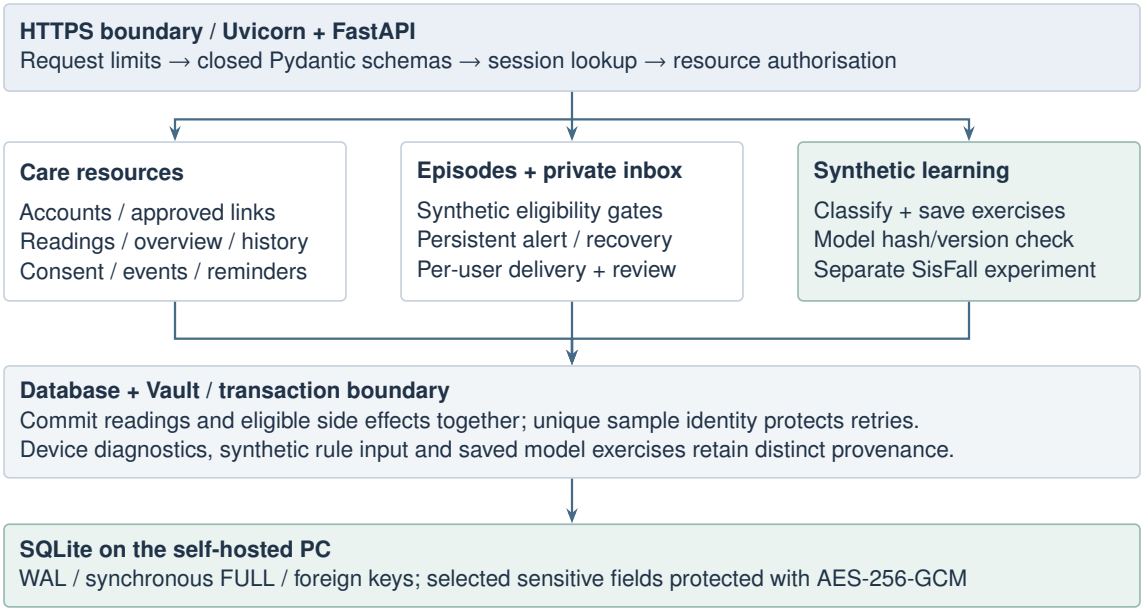


Figure 23. Server runtime responsibilities. Arrows show request/service dependencies and persistence, rather than a single diagnostic pipeline. Physical wearable envelopes are stored as diagnostics; the synthetic model does not classify surface-temperature measurements as core body temperature.

F.8 Server classes and function-based services

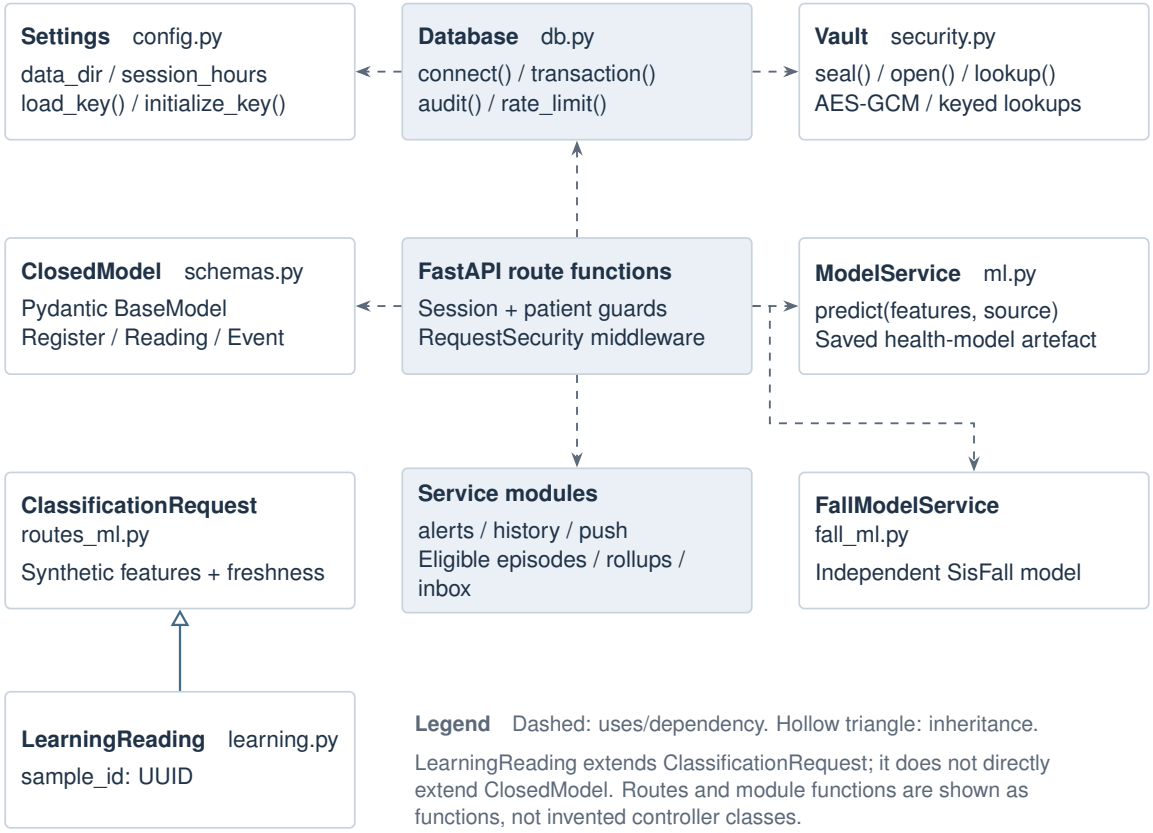
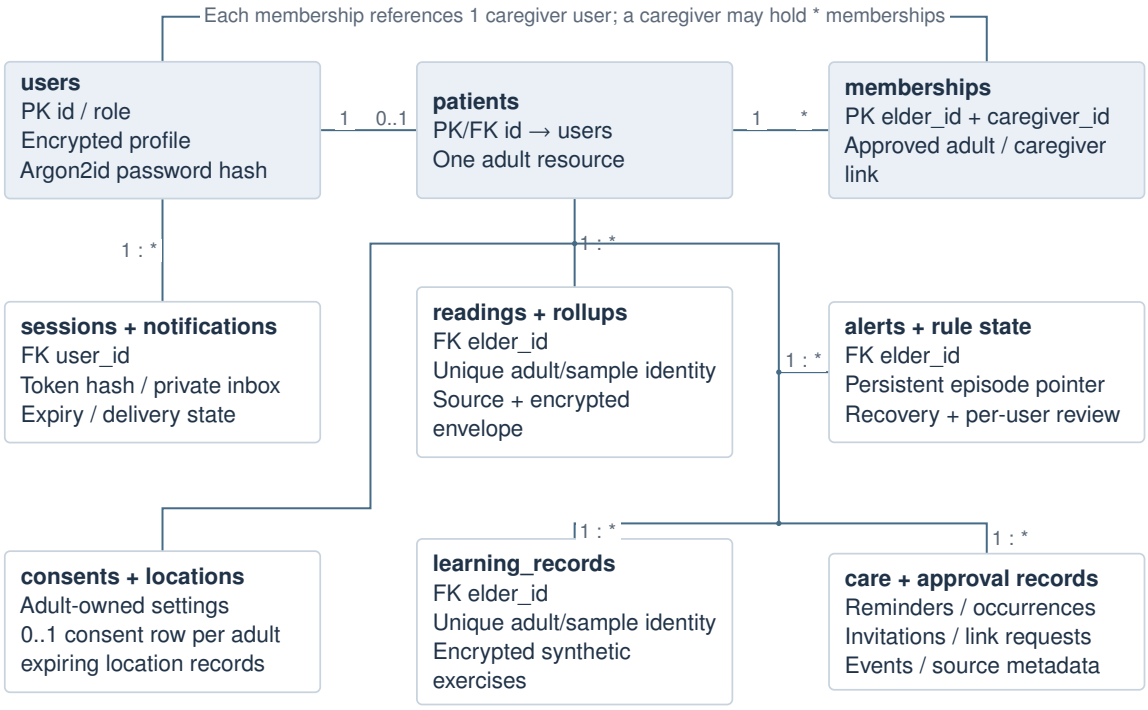


Figure 24. Selected server types and dependencies, derived from the named Python modules. The separate SisFall service remains an experimental model endpoint; it is not a live MPU6050-to-fall alarm pipeline.

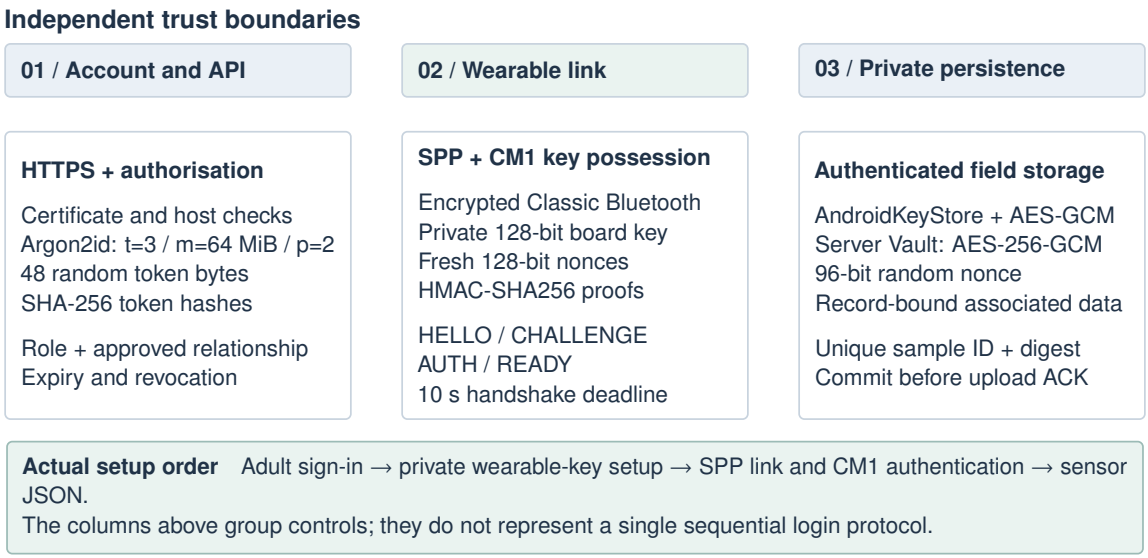
F.9 Persistent data domains



Grouped ownership map, not the full physical ERD. Patient-owned groups use elder_id; notifications also reference an adult resource. alert_acknowledgements uses (alert_id, user_id). Operational tables audit and rate_limits are omitted for clarity.

Figure 25. Principal SQLite ownership relationships from db.py. Cardinalities describe the database contracts, while grouped records retain their own keys and constraints. Selected fields are encrypted; indexed identifiers and timestamps remain visible metadata.

F.10 Authentication and encryption boundaries



Pairing without board input is not proof of resistance to an active pairing relay. Field encryption does not hide SQLite metadata. No passwords, bearer tokens, board keys or proof values appear in the report.

Figure 26. Implemented security controls across account, radio and storage boundaries. Session authorisation remains independent of Bluetooth key possession. One active phone owns the SPP link; telemetry follows successful CM1 authentication.

F.11 Capability interpretation for the current assembly

Table 17. Measured inputs, implemented behaviour and requested extensions. Clinical thresholds are not specified by this diagram set.

Area	Available input / behaviour	Claim boundary or next implementation
Pulse and oxygen	MAX30102 optical estimates with contact, calculation and validity flags	Estimates need stable optical coupling. Invalid values stay unavailable; no clinical accuracy claim follows from sensor presence.
Surface / ambient	MLX90614 object and local sensor temperature	Explain both channels. Ambient at the sensor is not a certified room measurement; surface is not core temperature. Current extreme-temperature rule demonstrations use synthetic input.
Wear detection	Optical-contact and sample-quality indicators	No contact can prompt a placement/connection check. No readings can also mean motion, disconnection or a fault; it is not a definitive not-worn state.
Walking / falling	MPU6050 acceleration and angular velocity	Current acquisition provides motion data. Step counting, real-time fall classification and a local fallback alarm are not implemented by this source. SisFall is an independent offline experiment.
Location	Phone GPS/location with consent, accuracy and expiry	Adult permission and server consent are required; caregivers see only approved shared data. Network and OS conditions affect availability.
Help and sound	Confirmed help event, durable private inbox and Android notification channel	Native posting can be measured. Audibility, hearing the alarm, response and emergency-service contact are different outcomes.
Buzzer fallback	GPIO25 output configured LOW	A fallback needs detection logic, an appropriate driver and independent power. No autonomous fall-triggered tone is established.
Hydration	Agreed hydration reminder in the care plan	These modules do not measure dehydration. A reminder is not a physiological assessment.
Energy	Phone handles GPS, uploads, storage and notifications	Offloading simplifies the wearable. Larger battery capacity alone does not establish longer runtime; current draw and endurance require measurement.

The component roles are checked against manufacturer descriptions for MAX30102 and MLX90614, and against the actual register acquisition in the firmware.[23, 24] Electrical labels describe breakout modules, not direct wiring to bare IC dies. The carrier's physical header order and supply selector must be checked on the actual KEYES revision; this source-derived map does not replace that inspection.